

Unit 5 String Processing and File Handling

5.1 Introduction to String

- ✓ A string is a sequence of characters.
- ✓ Strings can be created by enclosing characters inside a single quote or double-quotes.
- ✓ A string that contains no characters are called empty string.

Example:

<pre>s='Hello' print(s) s= "Hello" print(s) s = "Hello" print(s) s = """Hello, welcome to the world of Python""" print(s)</pre>	Output: <pre>Hello Hello Hello Hello, welcome to the world of Python</pre>
--	---

String Operation

Create list of strings

- ✓ To create a list of strings, first use square brackets [] to create a list.
- ✓ Then place the list items inside the brackets separated by commas.
- ✓ strings must be surrounded by quotes.
- ✓ use = to store the list in a variable.

Ex:

```
Name = ["Ami", "Neha", "Dharmesh"]
```

Ex:

```
Name = [
    "Ami ",
    " Neha ",
    " Dharmesh "
]
```

Print list of strings

- ✓ To print a whole list of strings in one line, you can simply call the built-in print function,

<pre>Name = ["Ami", "Neha", "Dharmesh"] print(Name)</pre>	Output <pre>['Ami', 'Neha', 'Dharmesh']</pre>
---	--

Add strings to list

- ✓ When you already have a list of strings and you want to add another string to it, you can use the append method:

<pre>Name = ["Ami", "Neha", "Dharmesh"]</pre>

```
Name.append("Aesha")
print(Name)
```

Concatenate lists of strings

- ✓ You can use the + operator to concatenate two lists of strings. For example:

```
Name1 = ["Ami", "Neha"]
Name2 = ["Aesha", "Monika"]
Name= Name1 + Name2
print(Name)
```

Output:
['Ami', 'Neha', 'Aesha', 'Monika']

Sort list of strings

- ✓ To sort a list of strings, you can use the sort method:

```
colors = ["red", "white", "blue"]
colors.sort()
print(colors)
```

Output:
['blue', 'red', 'white']

- ✓ You can also use the sorted() built-in function:

```
colors = ["red", "white", "blue"]
colors=sorted(colors)
print(colors)
```

Output:
['blue', 'red', 'white']

Join list of strings

- ✓ To join a list of strings by another string, you need to call the join method on the string, giving your list as an argument. For example, if we have this list:

```
colors = ["red", "white", "blue"]
print("".join(colors))
print(" ".join(colors))
print("|".join(colors))
```

Output:
redwhiteblue
red white blue
red|white|blue

5.2 Access String elements using index operator

- ✓ You can use a for loop, range in python and slicing operator to traverse a character in a string.

Using for loop to traverse a string

```
str="python"
for s in str:
    print(s)
```

Output:
p
y
t
h
o

	n
--	---

Using range to traverse a string:

<pre>str="python" for s in range(len(str)): print(str[s])</pre>	Output: p y t h o n
---	---------------------------------------

Using slicing operator to traverse a string partially

- ✓ A segment of a string is called a slice.
- ✓ Selecting a slice is similar to selecting a character:
- ✓ Subsets of strings can be taken using the slice operator (**[]** and **[:]**) with indexes starting at 0 in the beginning of the string and working their way from -1 at the end.
- ✓ Slice out substrings, sub lists, sub Tuples using index.

Syntax:[Start: stop: steps]

- ✓ Slicing will start from index and will go up to **stop** in **step** of steps.
- ✓ Default value of start is 0,
- ✓ Stop is last index of list
- ✓ And for step default is 1

<pre>str = 'Hello World!' print (str) print(str[0]) print (str[2:4]) print (str[2:]) print (str * 2)</pre>	Output Hello World! H ll llo World! Hello World!Hello World!
<pre>str="Scripting Language python" for s in str[0:6:1]: print(s)</pre>	Output S c r i p t
<pre>str="Scripting Language python" for s in str[:3]: print(s)</pre>	Output Sii na tn
<pre>str="python" for s in str[::-1]: print(s)</pre>	Output nohtyp
<pre>str="Python" print(str[::-1])</pre>	Output nohtyp

Using indexing ,print backward string

<pre>str="Python" l=len(str)-1 while l>=0: print(str[l]) l=l-1</pre>	<pre>Output n o h t y p</pre>
---	-------------------------------

5.3 String functions**• Basic functions: len, max, min**

len(): It returns number of characters in the string

syntax: variable name=len(string)

Example:

```
s="python"
```

```
l=len(s)
```

```
print(l)
```

Output:6

min(): It finds the smallest element in the sequence.

For one string, the character that has the smallest code is selected in the string.

For multiple string, a string is selected which, in lexicographic order (dictionary order), follows the first alphabetically.

Syntax:

```
ans=min(s)
```

Or

```
ans=min(s1,s2,...sn)
```

<pre>s="python" print(min(s)) s='abc+-0' print(min(s)) s='012345' print(min(s)) s='aAbB' print(min(s))</pre>	<pre>Output: h + 0 A</pre>
<pre>s=("def", "abc", "xyz") print(min(s)) s=('+', '-', '*', '/') print(min(s))</pre>	<pre>Output: abc *</pre>

max(): It finds the largest element in the sequence.

For one string, the character that has the largest code is selected in the string.

For multiple string, a string is selected which, in lexicographic order, follows the last alphabetically.

Syntax:

ans=max(s)

Or

ans=max(s1,s2,...sn)

<pre>s="python" print(max(s)) s='abc+-0' print(max(s)) s='012345' print(max(s)) s='aAbB' print(max(s))</pre>	Output: <pre>y c 5 b</pre>
<pre>s=("def","abc","xyz") print(max(s)) s=('+','-','*','/') print(max(s))</pre>	Output: <pre>xyz /</pre>

● **Testing functions: isalnum, isalpha, isdigit, isidentifier, islower, isupper, and isspace**

isalnum(): returns True if all the characters in the string is alphanumeric (a string which contains either number or alphabets or both). Otherwise return False

<pre>s=" " print(s.isalnum()) s="abc" print(s.isalnum()) s='123' print(s.isalnum()) s='abc123' print(s.isalnum()) s='+-' print(s.isalnum()) s='a b' print(s.isalnum())</pre>	Output: <pre>False True True True False False</pre>
--	--

isalpha: returns True if all the characters in the string are alphabets. Otherwise return False

<pre>s=" " print(s.isalpha()) s="abc" print(s.isalpha()) s='123' print(s.isalpha()) s='abc123' print(s.isalpha()) s='+-' print(s.isalpha())</pre>	Output: <pre>False True False False False</pre>
---	--

Isdigit: returns True if all the characters in the string are digits. Otherwise return False.

<pre>print("abc123".isdigit()) print("123".isdigit()) print("101 ".isdigit()) print("101.129".isdigit())</pre>	Output: False True False False
--	---

Identifier: returns True if string is an identifier or a keyword that is valid according to the python syntax Otherwise return False.

<pre>print("hello".isidentifier()) print("a1".isidentifier()) print("1a".isidentifier()) print("for".isidentifier())</pre>	Output: True True False True
--	---

islower: returns True if all the characters in the string are in lowercase. Otherwise return False.

<pre>s = 'Python' print(s.islower()) print("abc".islower())</pre>	Output: False True
---	---------------------------------

isupper: returns True if all the characters in the string are in uppercase. Otherwise return False.

<pre>s = 'PYthon' print(s.isupper()) print("ABC".isupper())</pre>	Output: False True
---	---------------------------------

isspace : returns True if all the characters in the string are whitespace characters. Otherwise return False.

<pre>print("\n\t".isspace()) print(" \n\t".isspace()) print("@ \n\t".isspace()) print("123".isspace())</pre>	Output: True True False False
--	--

● Searching functions: endswith, startswith, find, rfind, count

endswith: Syntax: endswith(suffix, start, end)

- ✓ Returns True when a string ends with the characters specified by suffix.
- ✓ You can limit the check by specifying a starting index using start or an ending index using end.
- ✓ If start and end indexes are not provided then, by default it takes 0 and length-1 as starting and ending indexes

<pre>s="abcdef" print(s.endswith('ef')) print("abcdef".endswith('ef',2,6)) print("abcdef".endswith('ef',2,4)) s1=('ab','cd','ef') s2="abcd 123 ef" print(s2.endswith(s1,0,len(s2)))</pre>	Output: True True False True
---	---

startswith: Syntax: startswith(prefix, start, end):

- ✓ Returns True when a string begins with the characters specified by prefix.
- ✓ You can limit the check by specifying a starting index using start or an ending index using end.
- ✓ If start and end indexes are not provided then, by default it takes 0 and length-1 as starting and ending indexes.

<pre>s="Hello" print(s.startswith('He')) print(s.startswith('lo')) print(s.startswith('He',0)) print(s.startswith('He',0,len(s))) print(s.startswith('He',2)) print("12345".startswith('123',0,4)) print("12345".startswith('123',2,4))</pre>	<pre>Output: True False True True False True False</pre>
---	--

Find: Syntax: find(str, start, end):

- ✓ Check whether str occurs in a string and outputs the index of the location.
- ✓ You can limit the search by specifying starting index using start or a ending index using end.
- ✓ If start and end indexes are not provided then, by default it takes 0 and length-1 as starting and ending indexes .
- ✓ If search not found then it returns -1.

<pre>s = 'Hello World' ans = s.find('Hello') print("Substring found at index:", ans) print(s.find('Wo')) print(s.find('l', 2)) print(s.find('a', 2)) print(s.find('o', 2, 10))</pre>	<pre>Output: Substring found at index: 0 6 2 -1 4</pre>
---	---

rfind: Syntax: rfind(str, start, end):

- ✓ Provides the same functionality as find(), but searches backward from the end of the string instead of the starting.
- ✓ You can limit the search by specifying starting index using start or a ending index using end.
- ✓ If start and end indexes are not provided then, by default it takes 0 and length-1 as starting and ending indexes .
- ✓ If search not found then it returns -1.

<pre>s = 'Hello World' ans = s.rfind('Hello') print("Substring found at index:", ans) print(s.rfind('Wo')) print(s.rfind('l', 2)) print(s.rfind('a', 2)) print(s.rfind('o', 2, 10))</pre>	<pre>Output: Substring found at index: 0 6 9 -1 7</pre>
--	---

count: Syntax: count(str, start, end):

- ✓ Counts how many times str occurs in a string.
- ✓ You can limit the search by specifying a starting index using start or an ending index using end.
- ✓ If start and end indexes are not provided then, by default it takes 0 and length-1 as starting and ending indexes.

s= "Hello Students Hello" print(s.count("Hello", 0, 5)) print(s.count("Hello", 0, 20)) print(s.count('l',0,20)) print(s.count('c',0,20))	Output: 1 2 4 0
--	-----------------------------

● Manipulation functions:

capitalize(): It returns a copy of the original string and converts the first character of the string to a capital (**uppercase**) letter, while making all other characters in the string **lowercase** letters.

Syntax: string_name.capitalize()

s= "hello world" print(s.capitalize()) print("heLLo".capitalize()) print('123'.capitalize())	Output: Hello world Hello 123
---	--

lower(): This function converts all uppercase characters in a string into lowercase characters and returns it.

Syntax: string.lower()

s= "AMI Patel" print("Original string:",s) print("Converted string:"+s.lower())	Output: Original string: AMI Patel Converted string:ami patel
---	---

upper (): This function converts all lowercase characters in a string into uppercase characters and returns it.

Syntax: string.upper()

s= "AMI Patel" print("Original string:",s) print("Converted string:"+s.upper())	Output: Original string: AMI Patel Converted string:AMI PATEL
---	---

title (): It converts the first character in each word to uppercase and the remaining characters to lowercase in the string and returns a new string.

Syntax: string.title()

s= "hELLo good morning" print("Original string:",s) print("Converted string:"+s.title())	Output: Original string: hELLo good morning Converted string:Hello Good Morning
--	---

swapcase: The string swapcase() method converts all uppercase characters to lowercase and lowercase characters to uppercase of the given string, and returns it.

Syntax: string.swapcase()

s= "hELLo GOOD morning" print("Original string:",s) print("Converted string:"+s.swapcase())	Output: Original string: hELLo GOOD morning Converted string: Hello good MORNING
---	--

replace: It returns a copy of the string where all occurrences of a substring are replaced with another substring.

Syntax: `string.replace(old_string, new_string, count)`

<pre>s="Hello World" s1=s.replace("l", "L") s2=s.replace("o","O",1) print(s) print(s1) print(s2)</pre>	Output: <pre>Hello World HeLLo WorLd HellO World</pre>
--	---

lstrip: It returns a copy of the string with leading characters removed (based on the string argument passed). If no argument is passed, it removes leading spaces.

Syntax: `string.lstrip([characters])`

Here, `characters [optional]`: A set of characters to remove as leading characters. By default, it uses space as the character to remove.

<pre>s=" Hello" print(s.lstrip()) s="++..--\$Hello" print(s.lstrip("+-"))</pre>	Output: <pre>Hello \$Hello</pre>
---	---

rstrip: It returns a copy of the string with trailing characters removed (based on the string argument passed). If no argument is passed, it removes trailing spaces.

Syntax: `string.rstrip([characters])`

Parameters: `characters (optional)` – a string specifying the set of characters to be removed.

<pre>s="Hello " print(s.rstrip()) s="Hello+..+&.." print(s.rstrip('+.&'))</pre>	Output: <pre>Hello Hello Hello</pre>
--	---

strip: It is an inbuilt function in the Python programming language that returns a copy of the string with both leading and trailing characters removed.

Syntax: `string.strip([characters])`

<pre>s=" Hello World..." print(s) print(s.strip()) print(s.strip('.'))</pre>	Output: <pre>Hello World... Hello World... Hello World</pre>
--	---

casefold: Python String `casefold()` method is used to convert string to lower case. It is similar to `lower()` string method, but case removes all the case distinctions present in a string.

- ✓ For example, the German letter ß is already lowercase so, the `lower()` method doesn't make the conversion.
- ✓ But the `casefold()` method will convert ß to its equivalent character ss.

Syntax: `string.casefold()`

<pre>s="HELLO" print(s.casefold()) print(s.lower())</pre>	Output: <pre>hello hello</pre>
---	---------------------------------------

● Formatting functions:

format: The format() method formats the specified value(s) and insert them inside the string's placeholder. The placeholder is defined using curly brackets: {}. It is used for handling complex string formatting more efficiently.

Syntax: string.format(value1, value2...)

a=5 b=10 s='{0}+{1}={2}'.format(a,b,a+b) print(s)	Output: 5+10=15
a=5 s='{0}**{1}={2}'.format(a,2,a**2) print(s)	Output: 5**2=25
s="My name is {fname}".format(fname = "abc") print(s)	Output: My name is abc

center: It creates and returns a new string that is padded with the specified character.

Syntax: string.center(length[, fillchar])

length: length of the string after padding with the characters.

fillchar: (optional) characters which need to be padded. If it's not provided, space is taken as the default argument.

s="Hello World" print(s.center(20)) print(s.center(5)) print(s.center(20,"#")) print(s.center(5,"#"))	Output: Hello World Hello World #####Hello World##### Hello World
---	---

ljust: This method left aligns the string according to the width specified and fills the remaining space of the line with blank space if 'fillchr' argument is not passed.

Syntax: ljust(len, fillchr)

len: The width of string to expand it.

fillchr (optional): The character to fill in the remaining space.

s = "hello" print(s.ljust(10)) print(s.ljust(10, '-')) print(s.ljust(10, '#'))	Output: hello hello----- hello#####
---	--

rjust: This method left aligns the string according to the width specified and fills the remaining space of the line with blank space if 'fillchr' argument is not passed.

Syntax: rjust(len, fillchr)

len: The width of string to expand it.

fillchr (optional): The character to fill in the remaining space.

s = "hello" print(s.rjust(10)) print(s.rjust(10, '-'))	Output: hello -----hello
--	--------------------------------

<code>print(s.rjust(10, '#'))</code>	<code>#####hello</code>
--------------------------------------	-------------------------

5.4 Introduction to Text files

- ✓ Python provides inbuilt functions for creating, writing, and reading files.
- ✓ There are two types of files that can be handled in python, normal text files and binary files (written in binary language, 0s, and 1s).
- ✓ Text files: In this type of file, Each line of text is terminated with a special character called EOL (End of Line), which is the new line character ('`\n`') in python by default.
- ✓ Binary files: In this type of file, there is no terminator for a line, and the data is stored after converting it into machine-understandable binary language.

✓ Difference between Text files and Binary files:

Text files	Binary files
Data stores in ASCII or Unicode .	Contains raw data
Content written in text files is human readable.	Content written in binary files is not human readable
Slower than binary file.	Faster than text file.
Text files are used to store data more user friendly.	Binary files are used to store data more compactly.

Files

- ✓ Files are named locations on disk to store related information.
- ✓ They are used to permanently store data in a non-volatile memory (e.g. hard disk).
- ✓ When we want to read from or write to a file, we need to open it first.
- ✓ When we are done, it needs to be closed so that the resources that are tied with the file are freed.
- ✓ In Python, a file operation takes place in the following order:
 - Open a file
 - Read or write (perform operation)
 - Close the file

5.5 File Handling functions:

● Basic functions: open, close

open File:

- ✓ Before performing any operation on the file like reading or writing, first, we have to open that file.
- ✓ For this, we should use Python's inbuilt function `open()`
- ✓ At the time of opening, we have to specify the mode, which represents the purpose of the opening file.

Syntax: `file object = open(filename, access mode)`

Example: `f = open("a.txt", "r")`

Where the following mode is supported:

<code>r</code>	open an existing file for a read operation.
----------------	---

w	open an existing file for a write operation. If the file already contains some data then it will be overridden but if the file is not present then it creates the file as well.
a	open an existing file for append operation. It will not override existing data.
r+	To read and write data into the file. The previous data in the file will be overridden
w+	To write and read data. It will override existing data.
a+	To append and read data from the file. It will not override existing data.

close File:

- ✓ close() function closes the file and frees the memory space acquired by that file.
- ✓ It is used at the time when the file is no longer needed or if it is to be opened in a different file mode.

Syntax: File object.close()

Example: f = open("a.txt", "r")
f.close()

● Reading file:

- ✓ To read a file in Python, we must open the file in reading (r) mode.
- ✓ There are three ways to read data from a text file.
read, readline and readlines.

read() : Returns the read bytes in form of a string. It reads n bytes, if no n specified, reads the entire file.

Syntax: File object.read([n])

f=open("a.txt","r") print(f.read()) f.close()	Output: hello everyone Good morning
--	---

readline() : Reads a line of the file and returns in form of a string. For specified n, reads at most n bytes. It does not reads more than one line.

Syntax: File object.readline([n])

f=open("a.txt","r") print(f.readline(5)) f.close()	Output: hello
--	------------------

Readlines() : Reads all the lines and return them as each line a string element in a list.

Syntax: File object.readlines()

f=open("a.txt","r") print(f.readlines()) f.close()	Output: ['hello everyone\n', 'Good morning\n', 'Have a nice day\n']
---	--

● writing file: write, append, writelines

- ✓ To write a file in Python, we must open the file in writing (w) mode or append mode(a).

write(): The write() method writes a string to a text file.

Syntax: File_object.write(str1)

Inserts the string str1 in a single line in the text file.

f=open("E:/a.txt","w") f.write("Hello World")	Output: Open a.txt file Hello World is display in text
--	--

f.close()	file.
-----------	-------

append: To append to a text file, you need to open the text file for appending mode.

When the file is opened in append mode, the handle is positioned at the end of the file. The data being written will be inserted at the end, after the existing data.

f=open("E:/a.txt","a") f.write("Good Morning") f.close()	Output: Open a.txt file Hello World Good Morning is display in text file.
--	--

writelines(): The writelines() method write a list of strings to a file at once.

Syntax: File_object.writelines(L) for L = [str1, str2, str3]

For a list of string elements, each string is inserted in the text file. Used to insert multiple strings at a single time.

f=open("E:/a.txt","w") lines=["Hello\n","good\n","morning"] f.writelines(lines) f.close()	Output: Open a.txt fileBelow content is displayed in text file. Hello good morning
--	---

Unit-4: Functions and Modules

4.1 Introduction to Python User defined Function

❖ What is a function in Python?

- ✓ A function is a block of code which only runs when it is called.
- ✓ In Python, **a function is a group of related statements that performs a specific task.**
- ✓ You can pass data, known as parameters; into a function.
- ✓ A function can return data as a result.
- ✓ Functions help break our program into smaller and modular chunks.
- ✓ As our program grows larger and larger, functions make it more organized and manageable.
- ✓ Furthermore, it avoids repetition and makes the code reusable.

Syntax of Function

```
def function_name (parameters):
```

```
    statement(s)
```

Above shown is a function definition that consists of the following components.

- ✓ **Keyword def** that marks the start of the function header.
- ✓ **A function name** to uniquely identify the function. Function naming follows the same rules of writing identifiers in Python.
- ✓ **Parameters** (arguments) through which we pass values to a function. They are optional.
- ✓ A colon (:) to mark the end of the function header.
- ✓ One or more valid python statements that make up the function body. **Statements must have the same indentation level.**
- ✓ return statement is used to return a value from the function.

❖ Example of creating a function:

```
def my_function():  
    print("Hello from a 21CE!!")
```

❖ How to call a function in python?

Once we have defined a function, we can call it from another function, program, or even the Python prompt. To call a function we simply type the function name with appropriate parameters.

❖ Example of call a function:

```
my_function()
```

Output:

```
Hello from a 21CE!!
```

4.2 Passing parameters to a function and returning values from a function

- ✓ Information can be passed into functions as arguments.
- ✓ Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.
- ✓ The following example has a function with one argument (name). When the function is called, we pass along a name, which is used inside the function to print the name:

Example of a function with parameter:

```
def my_function (name):  
  
    print ("Hello," + name + ". Good morning!")
```

calling a function:

```
my_function ('21CE')
```

Output:

Hello, 21CE. Good morning!

Returning a Values: To let a function return a value, use the return statement:

❖ Example of a function with parameter and return value:

```
def my_function(x):  
    return 5 * x
```

calling a function:

```
print (my_function(3))  
print (my_function(5))  
print (my_function(9))
```

Output:

```
15  
25  
45
```

Python Default Arguments:

- ✓ Function arguments can have default values in Python.
- ✓ We can provide a default value to an argument by using the assignment operator (=).

Example:

```
def sum(a,b=10):  
    return a+b  
print(sum(5))  
print(sum(5,5))
```

Output:

15
10

```
def my_function (name, msg="Good morning!"):  
    print("Hello ", name, msg)
```

```
my_function ("21CE")  
my_function ("ENGINEERS", "How do you do?")
```

Output:

Hello 21CE Good morning!
Hello ENGINEERS, How do you do?

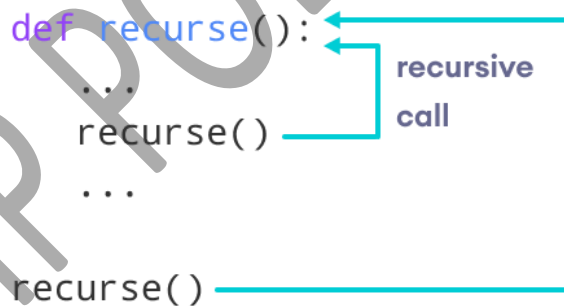
4.3 Recursion

❖ What is recursion?

- ✓ A Function calls itself is said to be a recursion.
- ✓ Recursion is the process of defining something in terms of itself.
- ✓ A physical world example would be to place two parallel mirrors facing each other. Any object in between them would be reflected recursively.

Python Recursive Function:

- ✓ In Python, we know that a function can call other functions. It is even possible for the function to call itself. These types of construct are termed as recursive functions.
- ✓ The following image shows the workings of a recursive function called recurse.

**Example of a recursive function to find the factorial of an integer.**

- ✓ Factorial of a number is the product of all the integers from 1 to that number. For example, the factorial of 6 (denoted as 6!) is $1*2*3*4*5*6 = 720$.

Example :

```
def factorial(x):
```

```
    if x == 1:
```



```
        return 1
```

```
    else:
```

```
        return (x * factorial(x-1))
```

calling a function:

```
num = 3
print("The factorial of", num, "is", factorial(num))
```

Output:

The factorial of 3 is 6

- ✓ Our recursion ends when the number reduces to 1. This is called the **base condition**.
- ✓ Every recursive function must have a base condition that stops the recursion or else the function calls itself infinitely.
- ✓ The Python interpreter limits the depths of recursion to help avoid infinite recursions, resulting in stack overflows.
- ✓ By default, the maximum depth of recursion is 1000. If the limit is crossed, it results in RecursionError.

Advantages of Recursion

- ✓ Recursive functions make the code look clean and elegant.
- ✓ A complex task can be broken down into simpler sub-problems using recursion.
- ✓ Sequence generation is easier with recursion than using some nested iteration.

Disadvantages of Recursion

- ✓ Sometimes the logic behind recursion is hard to follow through.
- ✓ Recursive calls are expensive (inefficient) as they take up a lot of memory and time.
- ✓ Recursive functions are hard to debug.

4.4 Standard Library: Built-in Functions

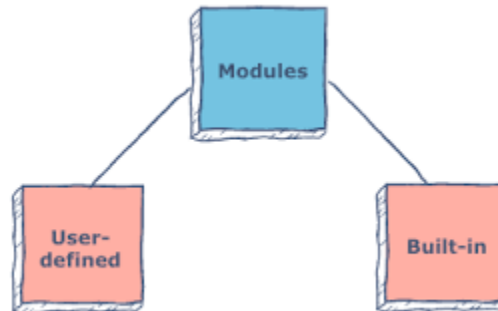
- ✓ A Python library is a collection of modules.
- ✓ A Package is a library that can be installed using a Package manager.
- ✓ Python standard libraries make the program simpler and remove the need to rewrite commonly used commands again and again.
- ✓ They can be used by calling/importing at the beginning of python script.

4.5 Modules and Packages

❖ What are modules in Python?

- ✓ Modules refer to a file containing Python statements and definitions.

- ✓ Modules are pre defined files that contain the python codes which include the basic functionality of class, methods ,variables etc.
- ✓ Modules can be divided in to two parts:



- ✓ Python contains built-in modules that are already programmed into the language and user defined modules are created by user.
- ✓ We can define our most used functions in a module and import it, instead of copying their definitions into different programs.
- ✓ A file containing Python code, for example: example.py, is called a module, and its module name would be **example**.

Example of user defined module (module name: example):

```
def add(a, b):  
    result = a + b  
    print( result)
```

- ✓ Here, we have defined a function add() inside a module named **example**. The function takes in two numbers and returns their sum.

How to import modules in Python?

- ✓ We use the import keyword to do this. To import our previously defined module example, we type the following in the Python prompt.

```
>>>import example
```

- ✓ This does not import the names of the functions defined in example directly in the current symbol table. It only imports the module name example there.
- ✓ Using the module name we can access the function using the dot (.) operator. For example:

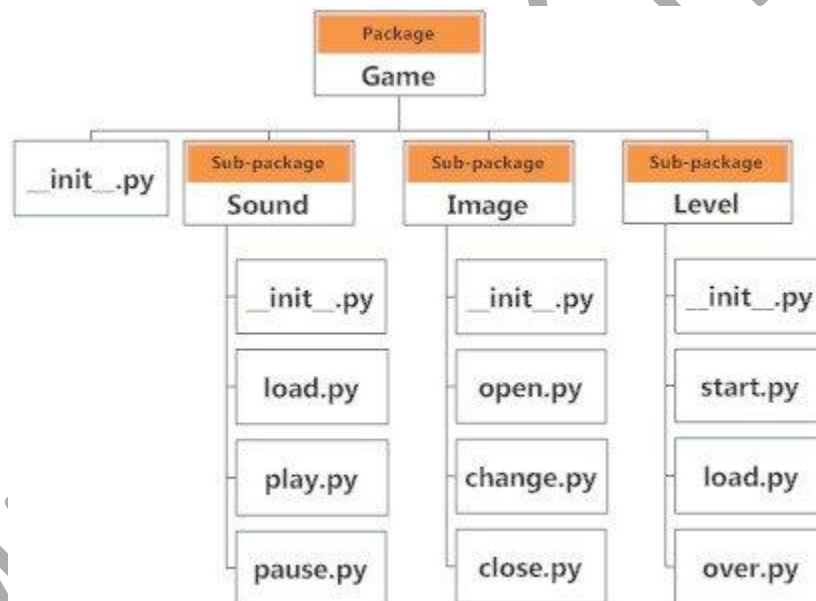
```
>>> example.add(4,5.5)
```

```
9.5
```

- ✓ Python has tons of standard modules. You can check out the full list of Python standard modules and their use cases. These files are in the Lib directory inside the location where you installed Python.

❖ What are packages?

- ✓ We don't usually store all of our files on our computer in the same location.
- ✓ We use a well-organized hierarchy of directories for easier access.
- ✓ **Python has packages for directories and modules for files.**
- ✓ **As a directory can contain subdirectories and files, a Python package can have sub-packages and modules.**
- ✓ A directory must contain a file named `__init__.py` in order for Python to consider it as a package. This file can be left empty but we generally place the initialization code for that package in this file.
- ✓ Here is an example. Suppose we are developing a **game**. One possible organization of packages and modules could be as shown in the figure below.



Importing module from a package

- ✓ We can import modules from packages using the dot (.) operator.
- ✓ For example, if we want to import the start module in the above example, it can be done as follows:

```
import Game.Level.start
```

Now, if this module contains a function named `select_difficulty()`, we must use the full name to reference it.

```
Game.Level.start.select_difficulty(2)
```

❖ rand module - Random numbers generators

- ✓ Python **Random module** is an in-built module of Python which is used to generate random numbers.
- ✓ It can be used to perform some action like to get a random number, select random element from a list, shuffle elements randomly etc.

Example: Printing a random value from a list

<pre>import random list1 = [1, 2, 3, 4, 5, 6] print(random.choice(list1))</pre>	Output: 2
--	---------------------

The random module has a set of methods, some of methods are listed here:

random(): It is used to generate random floats between 0 to 1. Syntax: random.random() Example: import random print(random.random()) Output: 0.37179335562307
uniform(): It is used to generate random floats between two given parameters. Syntax: random.uniform(First number,Second number) Example: import random print(random.uniform(1,10)) Output: 7.771509161751196
randint(): It returns a random integer between the given integers. Syntax: random.randint(First number,Second number) Example: import random print(random.randint(1,10)) Output: 3
randrange(): It returns a random integer from the range created by the start,stop and step arguments.The value of start is 0 by default. The value of step is 1 by default. Syntax: random. randrange (start,stop,step) Example: import random print(random.randrange(1,10,2)) Output: 5
choice():It returns a randomly selected element from a non empty sequence. Syntax: random.choice(sequence) Example: Selecting random elements from the list, string, and tuple Example 1: import random print(random.choice((1,10,2))) Output: 2 Example 2:import random print(random.choice('computer')) Output: o

choices(): It returns a list from a non empty sequence.

Syntax: random.choices(sequence)

Example: Selecting random elements from the list, string, and tuple

Example 1: import random
print(random.choices((1,10,2)))

Output: [2]

Example 2: import random
print(random.choices('computer'))

Output: ['o']

shuffle(): It is used to shuffle a sequence (list). Shuffling means changing the position of the elements of the sequence

Syntax: random.shuffle(sequence)

Example: import random
a=[10,11,12,13,14]
print('before shuffle',a)
random.shuffle(a)
print('after shuffle',a)

Output:
before shuffle [10, 11, 12, 13, 14]
after shuffle [10, 12, 14, 11, 13]

❖ math module – Mathematical functions

- ✓ Mathematical functions are defined in the math module.
- ✓ The math module has set of methods and constants.
- ✓ This module includes trigonometric functions , logarithm functions ,angle conversion functions etc.

Math constant:

Constant	Description	Example
math.e	Return Euler's number (2.718281828459045)	import math print(math.pi) print(math.e) print(math.tau)
math.pi	Return PI (3.141592653589793)	
math.tau	Return tau (6.283185307179586)	

Numeric Functions:

ceil(): Rounds a number up to the nearest integer

Example : import math
print(math.ceil(8.23))

Output : 9

floor(): Rounds a number down to the nearest integer

Example : import math
print(math.floor(8.23))

Output : 8

sqrt(): It returns the square root of the number.

Example: import math

<pre>print(math.sqrt(25))</pre> Output: 5
pow(): It receives two arguments, raises the first to the second and give result. Example: <pre>import math print(math.pow(2,4))</pre> Output: 16.0
factorial(): It returns the factorial of a given number. Example: <pre>import math print(math.factorial(4))</pre> Output : 24
gcd(): It is used to find the greatest common divisor of two numbers passed as the arguments. Example :<pre>import math print(math.gcd(10,35))</pre> Output :5
fabs(): It returns the absolute value of the number. Example :<pre>import math print (math.fabs(-10))</pre> Output : 10.0
fmod(): It returns the remainder of x/y. Example : <pre>import math print (math.fmod(10,3))</pre> Output : 1.0
exp(): It returns a float number after raising e to the power of a given number. (e**x) Example : <pre>import math print (math.exp(2))</pre> Output: 7.38905609893065

❖ Datetime module - Date and time functions

- ✓ Python Datetime module comes built into Python, so there is no need to install it externally.
- ✓ Python Datetime module supplies classes to work with date and time.
- ✓ The date and time objects can be either categorized as **aware or naïve** depending on whether it include timezone information.
- ✓ **Aware object** contains enough information to identify itself to other date/time objects.
- ✓ **naïve object** does not contain enough information to identify itself to other date/time objects.
- ✓ The DateTime module is categorized into 6 main classes –

date – An idealized naive date, assuming the current Gregorian calendar. Its attributes are year, month and day.
time – An idealized time, independent of any particular day, assuming that every day has exactly 24*60*60 seconds. Its attributes are hour, minute, second, microsecond, and tzinfo.
datetime – Its a combination of date and time along with the attributes year, month, day, hour, minute, second, microsecond, and tzinfo.
timedelta – A duration expressing the difference between two date, time, or datetime instances to microsecond resolution.
tzinfo – It provides time zone information objects.
timezone – A class that implements the tzinfo abstract base class as a fixed offset from the

```
UTC (New in version 3.2).
```

- ✓ We can use `dir()` function to get a list containing all attributes of a module.

```
import datetime  
print(dir(datetime))
```

Output

```
['MAXYEAR', 'MINYEAR', '__builtins__', '__cached__', '__doc__', '__file__',  
 '__loader__', '__name__', '__package__', '__spec__', 'date', 'datetime',  
 'datetime_CAPI', 'sys', 'time', 'timedelta', 'timezone', 'tzinfo']
```

Example:1 Get current date and time(Using datetime class)

```
import datetime  
d=datetime.datetime.now()  
print(d)
```

Output

```
2022-11-28 12:40:36.467347
```

Example:2 Get current date (Using date class)

```
import datetime  
d=datetime.date.today()  
print(d)
```

Output

```
2022-11-28
```

❖ matplotlib module – Plotting functions

- ✓ Matplotlib is a library for creating static, animated and interactive visualization in Python.

Installation steps for matplotlib:

Step 1: Make sure that Python and pip is preinstalled on system. Type following command in command prompt to check python is installed or not.

```
python -V
```

If python is successfully installed, then version of python will be displayed. To check pip, type following command in command prompt.

```
pip -V
```

If pip is successfully installed, then version of pip will be displayed

Step 2: Install Matplotlib.

Matplotlib can be installed using pip. Write following command to install Matplotlib.

```
pip install matplotlib
```

Step 3: Check if it is installed successfully.

To verify that Matplotlib is installed on your system, write following command in the command prompt. If it is successfully installed then version of Matplotlib will be displayed.

```
C:\Users\vpmp\AppData\Local\Programs\Python\Python310\Scripts > pip show matplotlib
```

For Importing matplotlib :

```
from matplotlib import pyplot as plt  
or  
import matplotlib.pyplot as plt
```

Types of plots in Matplotlib :

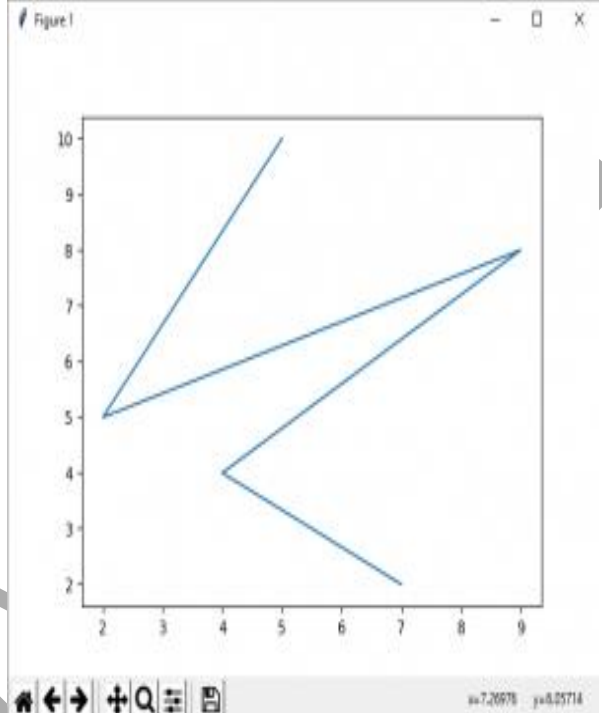
- ✓ Various plots can be created using Matplotlib like Bar Graph, Line Plot, Histogram, Scatter Plot, Area Plot etc.

1. Line plot :

EXAMPLE:

```
from matplotlib import pyplot as plt  
x = [5, 2, 9, 4, 7]  
y = [10, 5, 8, 4, 2]  
plt.plot(x,y)  
# function to show the plot  
plt.show()
```

OUTPUT:

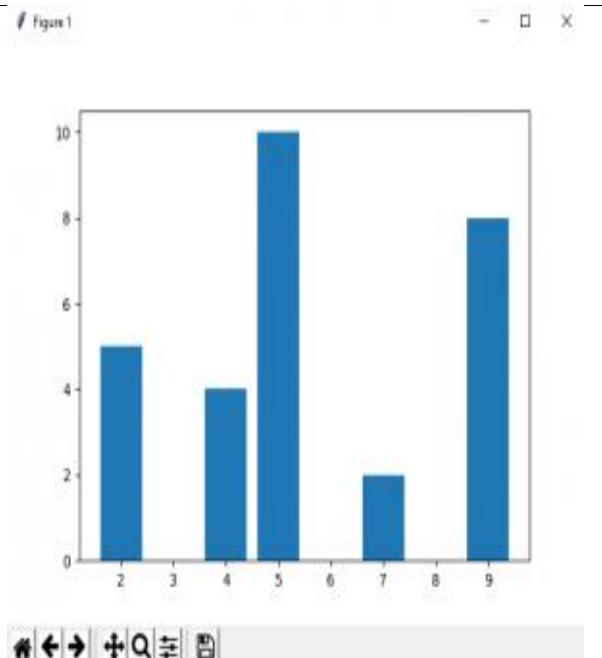


2. Bar plot : It uses bar to compare data.

EXAMPLE:

```
from matplotlib import pyplot as plt  
x = [5, 2, 9, 4, 7]  
y = [10, 5, 8, 4, 2]  
plt.bar(x,y)  
plt.show()
```

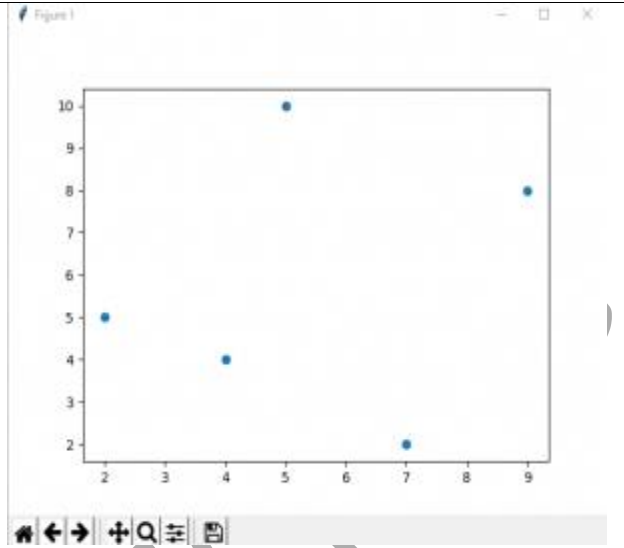
OUTPUT:



3. **Scatter Plot :** The data is displayed as a collection of points.

EXAMPLE:

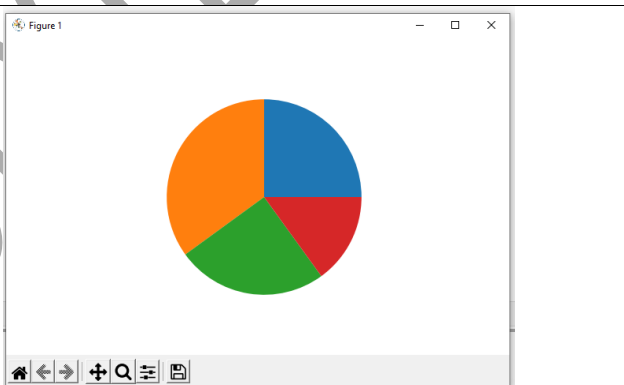
```
from matplotlib import pyplot as plt  
x = [5, 2, 9, 4, 7]  
y = [10, 5, 8, 4, 2]  
plt.scatter(x, y)  
plt.show()
```

OUTPUT:

4. **Pie Chart:** It refers to circular graph which is broken down into segments. It is used to show the percentage or proportional data where each slice of pie represents a category.

EXAMPLE:

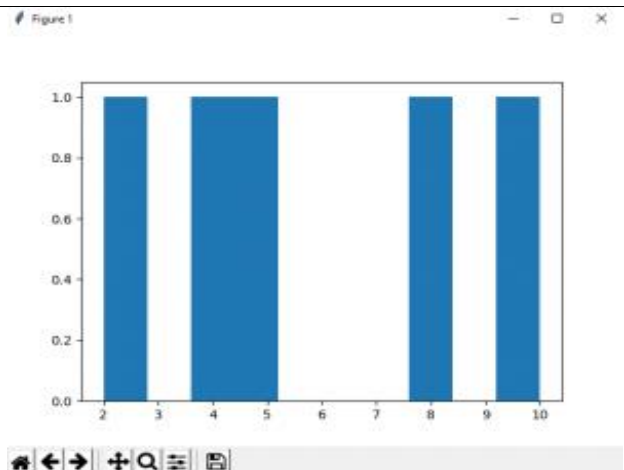
```
import matplotlib.pyplot as plt  
y=[25, 35, 25, 15]  
plt.pie(y)  
plt.show()
```

OUTPUT:

5. **Histogram :**

EXAMPLE:

```
from matplotlib import pyplot as plt  
y = [10, 5, 8, 4, 2]  
plt.hist(y)  
plt.show()
```

OUTPUT:

❖ Create and import custom user defined module

- ✓ We can define our most used functions in a module and import it, instead of copying their definitions into different programs.

Module Name: example

Python Module example

```
def add(a, b):  
    result = a + b  
    print(result)
```

Here, we have defined a function add() inside a module named example. The function takes in two numbers and print their sum.

How to import modules in Python?

- ✓ We can import the definitions inside a module to another module or the interactive interpreter in Python.
- ✓ We use the import keyword to do this. To import our previously defined module example, we type the following in the Python prompt.

```
import example  
example.add(4,5.5)
```

- ✓ This does not import the names of the functions defined in example directly in the current symbol table.
- ✓ It only imports the module name example there.
- ✓ Using the module name we can access the function using the dot (.) operator.
- ✓ Standard modules can be imported the same way as we import our user-defined modules.

❖ Various ways to import modules

Python import statement: We can import a module using the import statement and access the definitions inside it using the dot operator.

Example.

```
import math  
print("The value of pi is", math.pi)
```

Output:

The value of pi is 3.141592653589793

Python from...import statement: We can import specific names from a module without importing the module as a whole.

Example: import only pi from math module

```
from math import pi  
print("The value of pi is", pi)
```

Output:

The value of pi is 3.141592653589793

- Here, we imported only the pi attribute from the math module.

- In such cases, we don't use the dot operator. We can also import multiple attributes as follows:

```
from math import pi,e
print("The value of pi is", pi)
print("The value of e is", e)
```

Output:

```
The value of pi is 3.141592653589793
The value of e is 2.718281828459045
```

Import with renaming: We can import a module by renaming it as follows:

Example: import module by renaming it

```
import math as m
print("The value of pi is", m.pi)
```

Output:

```
The value of pi is 3.141592653589793
```

Import all names: We can import all names (definitions) from a module.

Example: import all names from the standard module math

```
from math import *
print("The value of pi is", pi)
```

Here, we have imported all the definitions from the math module. This includes all names visible in our scope except those beginning with an underscore(private definitions).

❖ Creating User Defined Package:

- ✓ Create a directory with the name EMP in home directory.
- ✓ Create a Python source file with name p1.py in EMP directory with following code.

```
def getName():
    name=['a','b','c']
    return name
```

- ✓ Now create one more python file with name p2.py in EMP directory with following code.

```
def getSalary():
    salary=[1000,2000,3000]
    return salary
```

- ✓ Now create __init__.py without any code in EMP directory.
- ✓ __init__.py converts Emp directory to Emp package with two modules p1 and p2.

❖ Importing a package in Python

- ✓ To use the module defined in Emp package, Create one file Test.py in home directory with code:

```
from Emp import p1
```

```
from Emp import p2
print(p1.getName())
print(p2.getSalary())
```

Now run Test.py to see the Output:

Output:['a', 'b', 'c']

[1000, 2000, 3000]

❖ PIP-Package Installer for Python:

- ✓ PIP is the package manager for Python packages.
- ✓ It is a package management system used to install and manage software packages written in Python.
- ✓ We use pip to install packages that do not come with Python.
- ✓ Python pip comes preinstalled on 3.4 or older version of Python.
- ✓ To check whether pip is installed or not, type below command in terminal. It shows the version of pip which is already installed in the system.

```
pip -version
```

- ✓ **Pip use PyPI as the default source for packages. So type following command on command prompt for installation of packages.**

```
pip install packagename
```

- ✓ Pip will look for that package on PyPI and if found ,download it and install the package on local system.
- ✓ For **uninstalling a package** type following command on command prompt.

```
pip uninstall packagename
```

Download and Install PIP:

- ✓ Download the get-pip.py file and store it in the same directory as python is installed.
- ✓ Change the current path of the directory in the command line to the path of the directory where get-pip.py file exists.
- ✓ Type following command in command prompt and wait through the installation process.

```
python get-pip.py
```

- ✓ Type following command to check whether pip is installed successfully or not. If successfully installed then display current version of it.

```
pip -V
```

Unit 3 Lists, Tuples, Sets and Dictionaries

❖ There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable, and unindexed. No duplicate members.
- **Dictionary** is a collection which is ordered and changeable. No duplicate members.

❖ Lists and operations on Lists

List: It is used to store the sequence of various types of data.

Characteristics of List:

- ✓ List is **ordered**.
- ✓ It allows **duplicate** members.
- ✓ It is a general purpose, most widely used in data structures.
- ✓ It is **mutable** type(We can **modify** its element after it is created)
- ✓ Elements of list can access by index.
- ✓ Python lists are one of the most versatile data types that allow us to work with multiple elements at once.
- ✓ To use a list, you must declare it first. It is declare using square brackets and separate values with commas.

Create Python Lists

- ✓ In Python, a list is created by placing elements inside square brackets [], separated by commas.

Example:

```
x = [1, 2, 3]
print(x)
```

Output:

```
[1, 2, 3]
```

- ✓ A list can have any number of items and they may be of different types (integer, float, string, etc.).

Example:

```
x = []
x = [1, "Neha", 3.4]
print(x)
```

Output:

```
[1, Neha, 3.4]
```

List operations:

- ✓ These operations include indexing, slicing, adding, multiplying, and checking for membership.
- ✓ We can also use + operator to combine two lists. This is also called concatenation.
- ✓ The * operator repeats a list for the given number of times.

Python Expression	Results	Description
<code>len([1, 2, 3])</code>	3	Length
<code>[1, 2, 3] + [4, 5, 6]</code>	<code>[1, 2, 3, 4, 5, 6]</code>	Concatenation
<code>['Hi!'] * 4</code>	<code>['Hi!', 'Hi!', 'Hi!', 'Hi!']</code>	Repetition
<code>3 in [1, 2, 3]</code>	True	Membership

Access List Elements

- ✓ There are various ways in which we can access the elements of a list.

List Index

- ✓ We can use the index operator `[ ]` to access an item in a list. In Python, indices start at 0. So, a list having 5 elements will have an index from 0 to 4.

```
x = ['h', 'e', 'l', 'l', 'o']

print(x[0])           # first character: h
print(x[2])           # third character: l
print(x[4])           # first character: o

# Nested List
x_list = ["Happy", [2, 0, 1, 3]]

print(x_list[0][1])   #a
print(x_list[1][3])   #3
```

Negative indexing

- ✓ Python allows negative indexing for its sequences. The index of -1 refers to the last item, -2 to the second last item and so on.

```
x = ['h','e','l','l','o']

print(x[-1])          # last item :o
print(x[-5])          # fifth last item:h
```

List Slicing

- ✓ We can access a range of items in a list by using the slicing operator.
- ✓ We can get sublist of the list using syntax:

List(start:stop:step)

Where start –starting index of the list

Stop—last index of the list

Step—skip the nth element within start and stop.

x=[2,4,6,8]		x=[2,4,6,8]	
print(x[:])	#Out put: [2, 4, 6, 8]	print(x[-1])	#Out put:8
print(x[1:])	# Out put: [4, 6, 8]	print(x[-3:])	#Out put: [4, 6, 8]
print(x[:2])	#Out put: [2, 4]	print(x[:-2])	#Out put: [2, 4]
print(x[1:3])	#Out put: [4, 6]	print(x[-3:-1])	#Out put: [4, 6]

x = ['h','e','l','l','o','w','o','r','l','d']	
print(x[2:5])	# elements from index 2 to index 4 : ['l', 'l', 'o']
print(x[5:])	# elements from index 5 to end: ['w', 'o', 'r', 'l', 'd']
print(x[:])	# elements beginning to end:['h', 'e', 'l', 'l', 'o', 'w', 'o', 'r', 'l', 'd']

x=[10,20,30,40,50,60,70]	
print(x[1:6:2])	#Out put: [20, 40, 60]

Add/Change List Elements

- ✓ Lists are mutable, meaning their elements can be changed unlike string or tuple.
- ✓ We can use the assignment operator = to change an item or a range of items.

x = [2, 4, 6, 8]	
x[0] = 1	# change the 1st item : [1, 4, 6, 8]
print(x)	
x[1:4] = [3, 5, 7]	# change 2nd to 4th items : [1, 3, 5, 7]
print(x)	

Append() and extend():

- ✓ We can add one item to a list using the append() method or add several items using the extend() method.

x = [1, 3, 5]	
x.append(7)	
print(x)	Output: [1, 3, 5, 7]
x.extend([9, 11, 13])	
print(x)	Output: [1, 3, 5, 7, 9, 11, 13]

Insert()

- ✓ We can insert one item at a desired location by using the method insert() or insert multiple items by squeezing it into an empty slice of a list.

```
x = [1, 9]
x.insert(1,3)

print(x)                Output: [1, 3, 9]

x[2:2] = [5, 7]

print(x)                Output: [1, 3, 5, 7, 9]
```

Delete

- ✓ We can delete one or more items from a list using the Python del statement. It can even delete the list entirely.

```
x = ['p', 'r', 'o', 'b', 'l', 'e', 'm']

del x[2]                # delete one item

print(x)                Output: ['p', 'r', 'b', 'l', 'e', 'm']

del x[1:5]              # delete multiple items

print(x)                Output: ['p', 'm']

del x                   # delete the entire list

print(x)                # Error: List not defined
```

Remove() and pop()

- ✓ We can use remove() to remove the given item or pop() to remove an item at the given index.
- ✓ The pop() method **removes and returns the last** item if the index is not provided

```
x = ['p','r','o','b','l','e','m']
x.remove('p')

print(x)                # Output: ['r', 'o', 'b', 'l', 'e', 'm']

print(x.pop(1))

print(x)                # Output: ['r', 'b', 'l', 'e', 'm']

print(x.pop())

print(x)                # Output: ['r', 'b', 'l', 'e']

x.clear()
```



```
print(x) # Output: []
```

Iterating Through a List

- ✓ Using a for loop we can iterate through each item in a list.

```
for name in ['Neha','Ami','Dharmesh']:
```

```
    print(name)
```

Output:

Neha

Ami

Dharmesh

Reverse a List: The reverse () method reverses the elements of the list.

```
x = ['h', 'e', 'l', 'l', 'o']
```

```
x.reverse()
```

```
print(x)
```

Output:

```
['o', 'l', 'l', 'e', 'h']
```

Sort List: The sort() method sorts the items of a list in ascending or descending order.

```
x = ['w', 'o', 'r', 'l', 'd']
```

```
x.sort()
```

```
print(x)
```

Output: ['d', 'l', 'o', 'r', 'w']

```
x=[10,2,56,3]
```

```
x.sort()
```

```
print(x)
```

Output: [2, 3, 10, 56]

```
x=[10,2,56,3]
```

```
x.sort(reverse=True)
```

```
print(x)
```

Output: [56, 10, 3, 2]

Python List Built in Function:

Function	Description	Example
len(list)	It is used to calculate the length of the list.	<pre>x=[10,20,30,40,50,60,70] print(len(x)) #Output:7</pre>
max(list)	It returns the maximum element of the list	<pre>x=[10,20,30,40,50,60,70] print(max(x)) #Output:70</pre>
min(list)	It returns the maximum element of the list	<pre>x=[10,20,30,40,50,60,70] print(min(x)) #Output: 10</pre>
list(seq)	It converts any sequence to the list.	<pre>str="abcd" print(type(str)) # <class 'str'> s=list(str) print(type(s)) # <class 'list'></pre>

❖ Tuples and operations on Tuples

Tuple: A tuple in Python is similar to a list.

- ✓ The difference between the two is that **we cannot change the elements of a tuple** once it is assigned whereas we **can change the elements of a list**.

Characteristics of Tuple:

- ✓ It is **ordered**.
- ✓ It allows **duplicate** members.

- ✓ It is **immutable** type(We cannot **modify** its element after it is created)
- ✓ Elements of Tuple can access by index.
- ✓ To use a tuple, you must declare it first. It is declare using () brackets and separate values with commas.
- ✓ A tuple can also be created without using parentheses. This is known as tuple packing

Creating a Tuple: We can construct tuple in many ways:

```
X=()      #no item tuple
X=(1,2,3)
X=tuple(list1)
X=1,2,3,4
```

Example:

Example 1: <pre>>>> x=(1,2,3) >>> print(x) Output: (1, 2, 3) >>> x Output: (1, 2, 3)</pre>	Example 2: <pre>>>> x=() >>> x Output: ()</pre>
Example 3: <pre>>>> x=[4,5,66,9] >>> y=tuple(x) >>> y Output: (4, 5, 66, 9)</pre>	Example 4: <pre>>>> x=1,2,3,4 >>> x Output: (1, 2, 3, 4)</pre>
Example 5: # tuple with mixed datatypes <pre>>>>x=(1,"hello",3.14) >>>print(x) Output: (1, 'hello', 3.14)</pre>	Example 6: nested tuple <pre>>>>x=("hello",[1,2,3],(4,5,6)) >>>x Output: ('hello', [1, 2, 3], (4, 5, 6))</pre>

- ✓ **A tuple can also be created without using parentheses. This is known as tuple packing.**

```
x = 1,3.14,"hello"
print(x)          # (1,3.14,'hello')
a, b, c = x        # tuple unpacking
print(a)          # 1
print(b)          # 3.14
print(c)          # hello
```

Operations on Tuples

- ✓ Access tuple items(indexing, negative indexing and Slicing)
- ✓ Change tuple items
- ✓ Loop through a tuple
- ✓ Count()
- ✓ Length()

Access tuple items(Indexing):

- ✓ We can use the index operator [] to access an item in a tuple, where the index starts from 0.
- ✓ So, a tuple having 6 elements will have indices from 0 to 5.

Example:

```
>>> x=('a','b','c','g')
>>> print(x[2])
```

Output: c**Example:**

```
x = ('Hello', [8, 4, 6], (1, 2, 3)) # nested tuple
print(x[0][3]) # nested index : 1
print(x[1][1]) # nested index : 4
```

Output:

```
1
4
```

Negative Indexing: Python allows negative indexing for its sequences. The index of -1 refers to the last item, -2 to the second last item and so on.

```
x = ('h', 'e', 'l', 'l', 'o')
```

```
print(x[-1])
```

```
print(x[-5])
```

```
print(x[-6])
```

Output:

```
o
```

```
h
```

```
Traceback (most recent call last):
```

```
File "<string>", line 5, in <module>
```

```
IndexError: tuple index out of range
```

```
>
```

Slicing

We can access a range of items in a tuple by using the slicing operator colon :

```
x=(2,4,6,8)
```

```
print(x[:]) #Out put: (2, 4, 6, 8)
```

```
print(x[1:]) # Out put: (4, 6, 8)
```

```
print(x[:2]) #Out put: (2, 4)
```

```
print(x[1:3]) #Out put: (4, 6)
```

```
x=(2,4,6,8)
```

```
print(x[-1]) #Out put:8
```

```
print(x[-3:]) #Out put: (4, 6, 8)
```

```
print(x[:-2]) #Out put: (2, 4)
```

```
print(x[-3:-1]) #Out put: (4, 6)
```

```
x = ('h','e','l','l','o','w','o','r','l','d')
```

```
print(x[1:4])
```

```
print(x[:-7])
```

```
print(x[7:])
```

```
print(x[:])
```

Output:

```
(e, 'l', 'l')
('h', 'e', 'l')
('r', 'l', 'd')
('h', 'e', 'l', 'l', 'o', 'w', 'o', 'r', 'l', 'd')
```

Change tuple items: Once a tuple is created, you cannot change its values. Tuples are unchangeable.

```
>>> x=(2,5,7,'4',8)
```

```
>>> x[1]=10
```

Output:

Traceback (most recent call last):

File "<pyshell#41>", line 1, in <module> x[1]=10

Example:

```
x = (4, 2, 3, [6, 5])
```

```
x[3][0] = 9
```

```
print(x)
```

```
x = ('h', 'e', 'l', 'l', 'o')
```

```
print(x)
```

Output:

```
(4, 2, 3, [9, 5])
```

```
('h', 'e', 'l', 'l', 'o')
```

Loop through a tuple(Iteration): We can loop the values of tuple using for loop

```
>>> x=4,5,6,7,2,'aa'
```

```
>>> for i in x:
```

```
    print(i)
```

Output:

```
4
```

```
5
```

```
6
```

```
7
```

```
2
```

```
aa
```

Length (): To know the number of items or values present in a tuple, we use len().

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
```

```
>>> y=len(x)
```

```
>>> print(y)
```

Output: 12

Concatenation: We can use + operator to combine two tuples. This is called concatenation.

- ✓ We can also repeat the elements in a tuple for a given number of times using the * operator.
- ✓ Both + and * operations result in a new tuple.

Example:

```
print((1, 2, 3) + (4, 5, 6))
print(("Repeat",) * 3)
```

Output:

```
(1, 2, 3, 4, 5, 6)
('Repeat', 'Repeat', 'Repeat')
```

Deleting a Tuple: we cannot change the elements in a tuple. It means that we cannot delete or remove items from a tuple.

Example:

```
x = (1,2,3)
```

```
del x
```

```
print(x)
```

Output:

Traceback (most recent call last):

File "<string>", line 8, in <module>

NameError: name 'x' is not defined

Tuple Methods

```
x = ('h', 'e', 'l', 'l', 'o',)
```

```
print(x.count('l'))
```

```
print(x.index('e'))
```

Output:

```
2
```

```
1
```

Python List Built in Function:

Function	Description	Example
len(tuple)	It is used to calculate the length of the tuple.	x=(10,20,30,40,50,60,70]) print(len(x)) #Output:7
max(tuple)	It returns the maximum element of the tuple	x=(10,20,30,40,50,60,70) print(max(x)) #Output:70
min(tuple)	It returns the maximum element of the tuple	x=(10,20,30,40,50,60,70) print(min(x)) #Output: 10
tuple (seq)	It converts any sequence to the tuple.	str="abcd" print(type(str)) # <class 'str'> s=tuple(str) print(type(s)) # <class 'tuple'>

❖ Where to use tuple?

- ✓ Tuple data is constant and must not be changed so when there is a requirement of read only data tuple is preferable.

- ✓ Tuple can simulate a dictionary without key. For example:
[(10,"abc",22), (20,"def",34), (30,"xyz",56)]

❖ Difference between List and Tuple

List	Tuple
It is created by []	It is created by ()
It is mutable	It is immutable
It has variable length.	It has fixed length.
It provides more functionality than a tuple.	It provides less functionality than a list.
It is used where the values of items can be changed.	It is used where we need to store the read only collections.
It is less memory efficient than a tuple.	It is more memory efficient than a list.
Example: a=[10,11,12]	Example: a=(10,11,12)

❖ Sets and operations on Sets

- ✓ **A set is an unordered collection of items.** Every set element is unique (no duplicates) and must be immutable (cannot be changed).
- ✓ Sets can also be used to perform mathematical set operations like union, intersection, symmetric difference, etc.
- ✓ We cannot access or change an element of a set using indexing or slicing. Set data type does not support it.

Characteristics of Set:

- ✓ Sets are unordered.
- ✓ Set element is unique. Duplicate elements are not allowed.
- ✓ Set are immutable.(Can not change value)
- ✓ There is no index in set. So they donot support indexing or slicing operator.
- ✓ The set are used for mathematical operation like union,intersection,difference etc.

Creating Python Sets

- ✓ A set is created by placing all the items (elements) inside curly braces {}, separated by comma, or by using the built-in set() function.
- ✓ It can have any number of items and they may be of different types (integer, float, tuple, string etc.).
- ✓ To make a set without any elements, we use the set() function without any argument.

Example:

Example 1: x = {1, 2, 3} print(x) Output: {1, 2, 3}	Example 2: # set of mixed datatypes x = {1.0, "Hello", (1, 2, 3)} print(x) Output: {1.0, (1, 2, 3), 'Hello'}
Example 3:	Example 4:

<pre># set cannot have duplicates x = {1, 2, 3, 4, 3, 2} print(x)</pre> Output: {1, 2, 3, 4}	<pre>x = set([1, 2, 3, 2]) print(x)</pre> Output: {1, 2, 3}
Example 5: <pre>x= set() print(type(x))</pre> Output: <class 'set'>	

Access value in a Set: We cannot access individual values in a set. We can only access all the elements together.

```
x = {1, 2, 3, 4, 3, 2}
for i in x:
    print(i,end=' ')
```

Output: 1 2 3 4

Modifying a set in Python

- ✓ We can add a single element using the `add()` method, and multiple elements using the `update()` method. The `update()` method can take tuples, lists, strings or other sets as its argument. In all cases, duplicates are avoided.

<pre>x = {1, 3} print(x)</pre>	Output: {1, 3}
<pre>x.add(2) print(x)</pre>	{1, 2, 3}
<pre>x.update([2, 3, 4]) print(x)</pre>	{1, 2, 3, 4}
<pre>x.update([4, 5], {1, 6, 8}) print(x)</pre>	{1, 2, 3, 4, 5, 6, 8}

Removing elements from a set

- ✓ A particular item can be removed from a set using the methods `discard()` and `remove()`.
- ✓ The only difference between the two is that **the `discard()` function leaves a set unchanged if the element is not present in the set.** On the other hand, the **`remove()` function will raise an error** in such a condition (if element is not present in the set).

<pre>x = {1, 3, 4, 5, 6} print(x) x.discard(4) print(x) x.remove(6) print(x) x.discard(2) print(x) x.remove(2)</pre>	<pre>Output: {1, 3, 4, 5, 6} {1, 3, 5, 6} {1, 3, 5} {1, 3, 5} Traceback (most recent call last): File "<string>", line 28, in <module> KeyError: 2</pre>
--	--

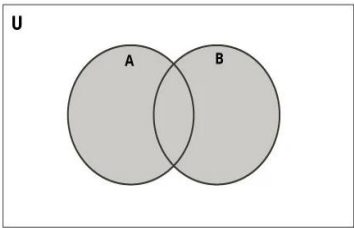
- ✓ Similarly, we can remove and return an item using the pop() method.
- ✓ Since set is an unordered data type, there is no way of determining which item will be popped.
- ✓ We can also remove all the items from a set using the clear() method.

<pre>x = set("abcdefgh") print(x) print(x.pop()) print(x.pop()) print(x) x.clear() print(x)</pre>	<pre>Output: {'d', 'c', 'b', 'a', 'f', 'h', 'e', 'g'} d c {'b', 'a', 'f', 'h', 'e', 'g'} set()</pre>
--	--

Python Set Operations

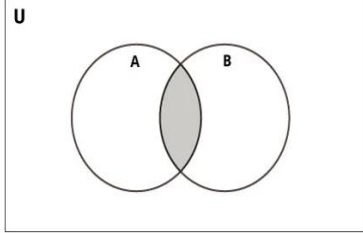
- ✓ Sets can be used to carry out mathematical set operations like union, intersection, difference and symmetric difference. We can do this with operators or methods.

Set Union: Union of A and B is a set of all elements from both sets. Union is performed using | operator. Same can be accomplished using the union() method.

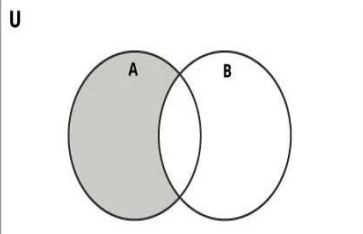
	<pre>A = {1, 2, 3, 4, 5} B = {4, 5, 6, 7, 8} print(A B)</pre> <pre>Output: {1, 2, 3, 4, 5, 6, 7, 8}</pre>	<pre>A = {1, 2, 3, 4, 5} B = {4, 5, 6, 7, 8} print(A.union(B)) print(B.union(A))</pre> <pre>Output: {1, 2, 3, 4, 5, 6, 7, 8} {1, 2, 3, 4, 5, 6, 7, 8}</pre>
---	---	---

Set Intersection: Intersection of A and B is a set of elements that are common in both the sets.

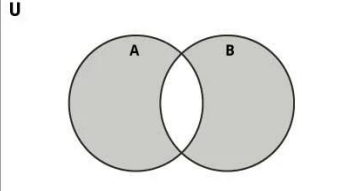
Intersection is performed using & operator. Same can be accomplished using the intersection() method.

	<pre>A = {1, 2, 3, 4, 5} B = {4, 5, 6, 7, 8} print(A & B)</pre> Output: {4, 5}	<pre>A = {1, 2, 3, 4, 5} B = {4, 5, 6, 7, 8} print(A.intersection(B)) print(B.intersection(A))</pre> Output: {4, 5}
---	---	--

Set Difference: Difference of the set B from set A ($A - B$) is a set of elements that are only in A but not in B. Similarly, $B - A$ is a set of elements in B but not in A. Difference is performed using - operator. Same can be accomplished using the difference() method.

	<pre>A = {1, 2, 3, 4, 5} B = {4, 5, 6, 7, 8} print(A - B)</pre> Output: {1,2,3}	<pre>A = {1, 2, 3, 4, 5} B = {4, 5, 6, 7, 8} print(A.difference(B)) print(B.difference(A))</pre> Output: {1,2,3} {6,7,8}
---	--	---

Set Symmetric Difference: Symmetric Difference of A and B is a set of elements in A and B but not in both (excluding the intersection). Symmetric difference is performed using ^ operator. Same can be accomplished using the method symmetric_difference().

	<pre>A = {1, 2, 3, 4, 5} B = {4, 5, 6, 7, 8} print(A ^ B)</pre> Output: {1,2,3,6,7,8}	<pre>A = {1, 2, 3, 4, 5} B = {4, 5, 6, 7, 8} print(A.symmetric_difference(B)) print(B.symmetric_difference(A))</pre> Output: {1,2,3,6,7,8}
---	--	---

Set Membership Test: We can test if an item exists in a set or not, using the in keyword.

<pre>x = set("apple") print('a' in x) print('p' not in x)</pre>	Output: True False
---	-----------------------------------

Iterating Through a Set: We can iterate through each item in a set using a for loop.
Example:

<pre>for letter in set("apple"): print(letter)</pre>	Output: a p l e
--	--

Built-in Functions with Set

<code>len()</code>	Returns the length (the number of items) in the set.
<code>max()</code>	Returns the largest item in the set.
<code>min()</code>	Returns the smallest item in the set.
<code>sorted()</code>	Returns a new sorted list from elements in the set(does not sort the set itself).
<code>sum()</code>	Returns the sum of all elements in the set.

<pre>A = {1, 5, 3, 2, 4} print(len(A)) print(min(A)) print(max(A)) print(sorted(A)) print(sum(A))</pre>	<pre>Output: 5 1 5 [1, 2, 3, 4, 5] 15</pre>
---	---

❖ Dictionaries and operations on Dictionaries

- ✓ Python dictionary is an ordered collection of items. Each item of a dictionary has a key/value pair.

Characteristics of Dictionary:

- ✓ It is used to store data in a key-value pair format.
- ✓ It is mutable(changeable).
- ✓ Duplicate values are not allowed.
- ✓ Key must be a single element.
- ✓ Value can be of any type such as list,tuple,integer etc.

Creating Python Dictionary: Creating a dictionary is as simple as placing items inside **curly braces {}** separated by commas.

An item has a key and a corresponding value that is expressed as a pair (key: value). We can also create a dictionary using the **built-in dict() function**.

<pre>x = {} print(x)</pre>	<pre>Output {} </pre>
<pre>x = {1: 'abc', 2: 'def'} # dictionary with integer keys print(x)</pre>	<pre>{1: 'abc', 2: 'def'}</pre>
<pre>x = {'name': 'abc', 1: 'def'} print(x)</pre>	<pre>{'name': 'abc', 1: 'def'}</pre>
<pre>x = dict({1:'abc', 2:'def'}) # using dict() print(x)</pre>	<pre>{1: 'abc', 2: 'def'}</pre>
<pre>x = dict([(1,'abc'), (2,'def')])</pre>	<pre>{1: 'abc', 2: 'def'}</pre>

print(x)	
----------	--

Accessing Elements from Dictionary:

- ✓ While indexing is used with other data types to access values, a dictionary uses keys.
- ✓ Keys can be used either inside square brackets [] or with the get() method.

<pre>x = {'name': 'abc', 'rollno': 1} print(x['name']) print(x.get('rollno')) print(x.get('address')) # Output None print(x['address']) # Trying to access keys which doesn't exist throws error</pre>	Output abc 1 None Traceback (most recent call last): File "<string>", line 9, in <module> KeyError: 'address'
---	---

Changing and Adding Dictionary elements:

- ✓ We can add new items or change the value of existing items using an **assignment operator**.
- ✓ If the key is already present, then the existing value gets updated. In case the key is not present, a new (key: value) pair is added to the dictionary.

<pre>x = {'name': 'abc', 'rollno': 1} print(x) x['rollno'] = 20 # update value print(x) x['address'] = 'xyz' # adding value print(x)</pre>	Output {'name': 'abc', 'rollno': 1} {'name': 'abc', 'rollno': 20} {'name': 'abc', 'rollno': 20, 'address': 'xyz'}
--	---

Removing elements from Dictionary:

- ✓ We can remove a particular item in a dictionary by using the **pop() method**. This method removes an item with the provided key and returns the value.
- ✓ The **popitem()** method can be used to remove and return an arbitrary (key, value) item pair from the dictionary.
- ✓ All the items can be removed at once, using the **clear()** method.
- ✓ We can also use the **del keyword** to remove individual items or the entire dictionary itself.

Using pop() squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25} print(squares.pop(4)) # remove a particular item, returns its value print(squares)	Output: 16 {1: 1, 2: 4, 3: 9, 5: 25}
Using popitem() squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25} print(squares.popitem()) # remove an arbitrary item, return (key,value) print(squares)	Output: (5, 25) {1: 1, 2: 4, 3: 9, 4: 16}

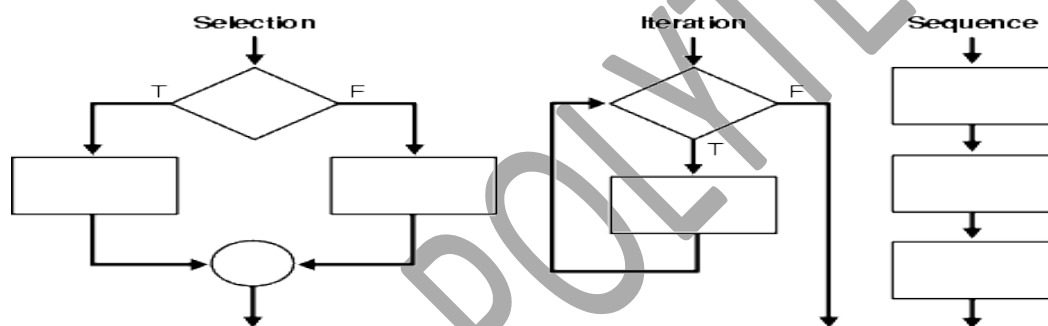
Using clear() squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25} squares.clear() print(squares)	Output: { }
Using del keyword squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25} del squares[2] print(squares) del squares print(squares)	Output: {1: 1, 3: 9, 4: 16, 5: 25} NameError: name 'squares' is not defined

Unit 2 Control Flow Structures

❖ Introduction to Control Structures

- ✓ A control structure (or flow of control) is a block of programming that analyses variables and chooses a direction in which to go based on given parameters.
- ✓ A control structure is just a decision that the computer makes. So, it is the basic **decision-making process** in programming and flow of control determines how a computer program will respond when given certain conditions and parameters.
- ✓ There are two basic aspects of computer programming: **data** and **instructions**.
- ✓ To work with data, you need to understand variables and data types;
- ✓ to work with instructions, you need to understand control structures and statements.
- ✓ **Flow of control** through any given program is implemented with three basic types of control structures: Sequential, Selection and Repetition.
- ✓ There are three types of **Python control structures**.

1. **Sequential**
2. **Selection**
3. **Repetition (iteration).**



1. **Python Sequential:** Flow of a program that executes in an order, without skipping, jumping or switching to another block of code. You cannot execute the second instruction before executing the instruction above it.
2. **Python Iteration:** Iteration repeats a body of code as long as condition is true. Python provides while and for loop for iteration.
3. **Python Selection:** Selection allows programmer to ask questions, based on the result, perform different action.

❖ **if Statement :** In this control structure statement execute only if the result of condition is true .

Flowchart	Syntax
	<pre>if test expression: statement(s)</pre>

- ✓ The program evaluates the test expression. If the test expression is True, then statement(s) will be executed.
- ✓ If the test expression is False, the statement(s) is not executed.
- ✓ In Python, the body of the if statement is indicated by the indentation. The body starts with an indentation and the first unindented line marks the end.
- ✓ Python interprets non-zero values as True. None and 0 are interpreted as False.

Example:

Example: Python if Statement

```
num = 3
if num > 0:
    print(num, "is a positive number.")
print("This is always printed.")
num = -1
if num > 0:
    print(num, "is a positive number.")
print("This is also always printed.")
```

Output:

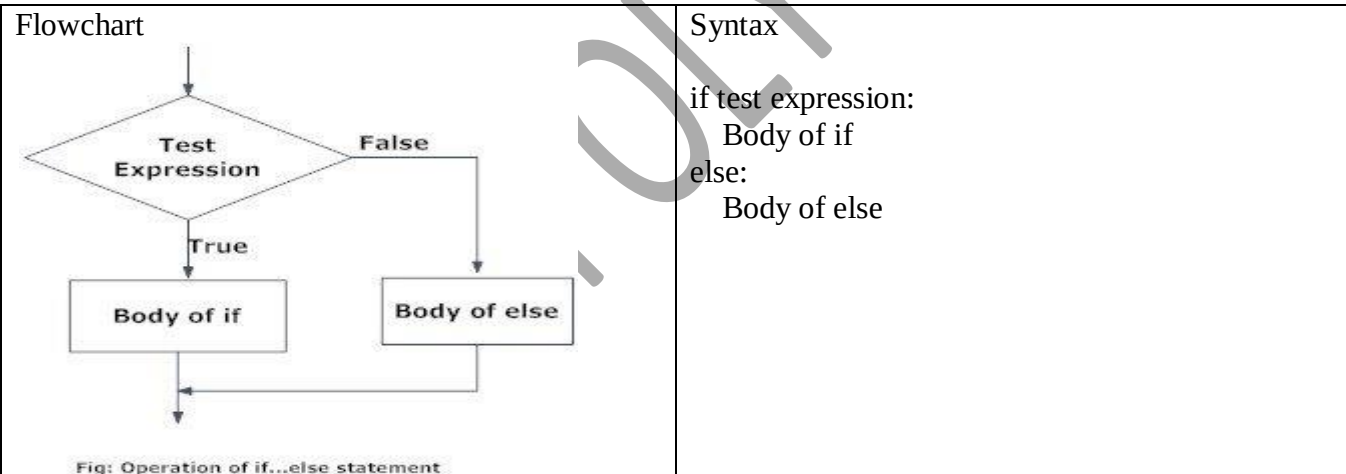
3 is a positive number.
This is always printed.
This is also always printed.

```
a = 3
if a > 2:
    print(a, "is greater")
print("done")
a = -1
if a < 0:
    print(a, "is smaller")
print("Finish")
```

Output:

3 is greater
done
-1 is smaller
Finish

- ❖ **if else Statement** : When we have to select only one option from the given two options, then we use if ...else.



- ✓ The if..else statement evaluates test expression and will execute the body of if only when the test condition is True.
- ✓ If the condition is False, the body of else is executed. Indentation is used to separate the blocks.

Example: Python if...else Statement

```
num = 3
if num >= 0:
    print("Positive or Zero")
else:
    print("Negative number")
```

Output:

Positive or Zero

```
a = 7
b = 0
if (a > b):
    print("a is greater than b")
else:
    print("b is greater than a")
```

Output:

a is greater than b

- ❖ **Nested if-else :** Nested “if-else” statements mean that an “if” statement or “if-else” statement is present inside another if or if-else block.

Syntax

```

if ( test condition 1):
    if ( test condition 2):
        Statement1
    else:
        Statement2
else:
    Statement 3

```

- ✓ First test condition1 is checked, if it is true then check condition2.
- ✓ If test condition 2 is True, then statement1 is executed.
- ✓ If test condition 2 is false, then statement2 is executed.
- ✓ If test condition 1 is false, then statement3 is executed

Example: Python Nested if...else Statement

```

num = int(input("Enter a number: "))
if num >= 0:
    if num == 0:
        print("Zero")
    else:
        print("Positive number")
else:
    print("Negative number")

```

Output:

Enter a number: 12
Positive number

```

a=int(input("Enter A: "))
b=int(input("Enter B: "))
c=int(input("Enter C: "))
if a>b:
    if a>c:
        max=a
    else:
        max=c
else:
    if b>c:
        max=b
    else:
        max=c

```

print("Maximum = ",max)

Output:

Enter A: 23
Enter B: 2
Enter C: 56
Maximum = 56

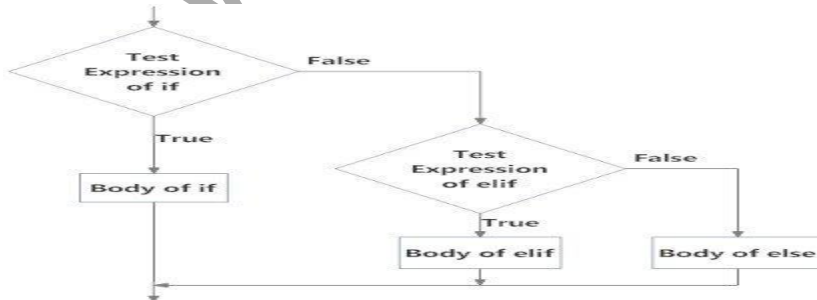
❖ **if-elif-else statements****Flowchart**

Fig: Operation of if...elif...else statement

Syntax

```

if test expression:
    Body of if
elif test expression:
    Body of elif
else:
    Body of else

```

- ✓ The elif is short for else if. It allows us to check for multiple expressions.
- ✓ If the condition for if is False, it checks the condition of the next elif block and so on.
- ✓ If all the conditions are False, the body of else is executed.
- ✓ Only one block among the several if...elif...else blocks is executed according to the condition.
- ✓ The if block can have only one else block. But it can have multiple elif blocks.

Example: Python if-elif-else Statement

<pre>num = -3 if num > 0: print("Positive number") elif num == 0: print("Zero") else: print("Negative number")</pre> Output: Negative number	<pre>a,b,c=12,45,3 if a>b and a>c: print("a is max") elif b>a and b>c: print("b is max") else: print("c is max")</pre> Output: b is max
--	--

❖ Switch Statement

- ✓ Python Switch case is a selection control statement.
- ✓ The switch expression evaluated once. The value of the expression is compared with the value of each case.
- ✓ If there is a match, the associated block of code is executed.
- ✓ Python does not have its inbuilt switch case statement.
- ✓ We have to implement using different methods.
 - Using Function and Lambda Function
 - Using Dictionary Mapping
 - Using Python classes
 - Using if-elif-else

Using Dictionary Mapping:

```
def monday():
    return "monday"
def tuesday():
    return "tuesday"
def wednesday():
    return "wednesday"
def thursday():
    return "thursday"
def friday():
    return "friday"
def saturday():
    return "saturday"
def sunday():
    return "sunday"
def default():
    return "Incorrect day"

Day = {
```



```

1: monday,
2: tuesday,
3: wednesday,
4: thursday,
5: friday,
6: saturday,
7: sunday
}

```

```

def DayName(dayofWeek):
    return Day.get(dayofWeek,default)( )

```

```

print(DayName (3))
print(DayName (8))

```

Output:

Wednesday

Incorrect day

- ✓ First create different functions which return Day name.
- ✓ We create a dictionary in which stored different key-value pair.
- ✓ After that create main function which takes user input for finding day name.
- ✓ At last, called the function to print the desired output

❖ loop

- ✓ A loop statement allows us to execute a statement or group of statements multiple times as long as the condition is true.
- ✓ Repeated execution of a set of statements with the help of loops is called iteration.
- ✓ Loops statements are used when we need to run same code again and again, each time with a different value.
- ✓ In Python Iteration (Loops) statements are of three types:
 1. While Loop
 2. For Loop
 3. Nested For Loops

❖ while loop

Flowchart

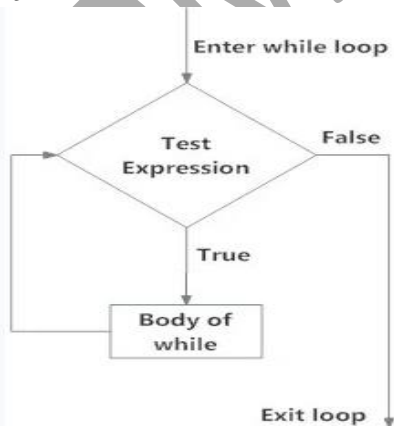


Fig: operation of while loop

Syntax

```

while test_expression:
    Body of while

```

- ✓ Loops are either infinite or conditional.
- ✓ In the while loop, test expression is checked first.
- ✓ The body of the loop is entered only if the test_expression evaluates to True.
- ✓ After one iteration, the test expression is checked again. This process continues until the test_expression evaluates to False.
- ✓ The statements that are executed inside while can be a single line of code or a block of multiple statements

Example:

```
x=1
while(x<=5):
    print(x)
    x+=1
```

Output:

1
2
3
4
5

```
a = 5
b = 1
while b <= 5:
    print (a,"*",b,"=",a*b)
    b+=1
```

Output:

5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25

❖ **for loop**

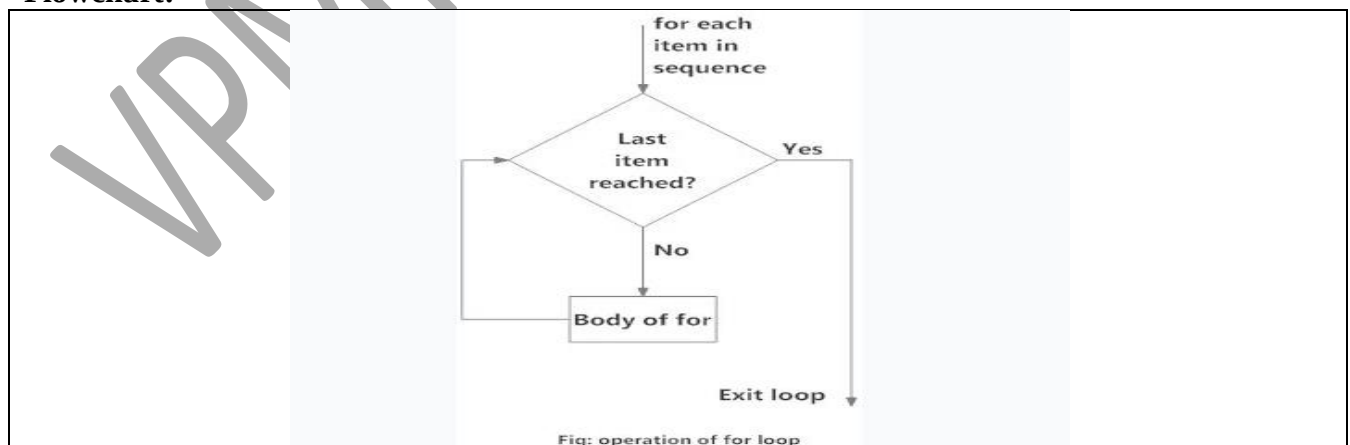
- ✓ Python for loop is used for repeated execution of a group of statements for the desired number of times.
- ✓ It iterates over the items of lists, tuples, strings, the dictionaries and other iterable objects

Syntax: for var in sequence:

Statement(s)

Holds the value of item
in sequence in each iteration

A sequence of values assigned to var in each iteration

Flowchart:**Example:**

<pre>for i in range(1,6): print(i)</pre> Output: 1 2 3 4 5	<pre>a=[10,12,13,14] for i in a: print(i)</pre> Output: 10 12 13 14
sum of 1 to 5 no <pre>sum=0 for i in range(1,6): sum=sum+i print(sum)</pre> Output: 15	Print Multiplication table <pre>b=1 a=int(input("enter n:")) for b in range(1,6): print (a,"*",b,"=",a*b)</pre> Output: enter n:4 4 * 1=4 4 * 2=8 4 * 3=12 4 * 4=16 4 * 5=20
<pre>n=int(input ("enter n:")) fact=1 for i in range(1,n+1): fact=fact*i print ("factorial=",fact)</pre> Output: enter n:3 factorial=6	<pre>n=int(input ("enter n:")) n1=0 n2=1 print (n1) print(n2) for i in range(3,n+1): n3=n1+n2 print(n3) n1=n2 n2=n3</pre> Output: 0 1 1 2 3 5

❖ Nested loops

- ✓ In Python programming language there are two types of loops which are for loop and while loop.
- ✓ Using these loops we can create nested loops in Python.
- ✓ Nested loops mean loops inside a loop. For example, while loop inside the for loop, for loop inside the for loop, etc.

Syntax:	Syntax:
for var in sequence:	while expression:

```
for var in sequence:
    statements(s)
statements(s)
```

```
while expression:
    statement(s)
statement(s)
```

Example :1

```
rows = 5
for i in range(1, rows + 1):
    for j in range(1, i + 1):
        print('*',end=' ')
    print(' ')
```

Output:

```
*
* *
* * *
* * * *
* * * * *
```

Example :2

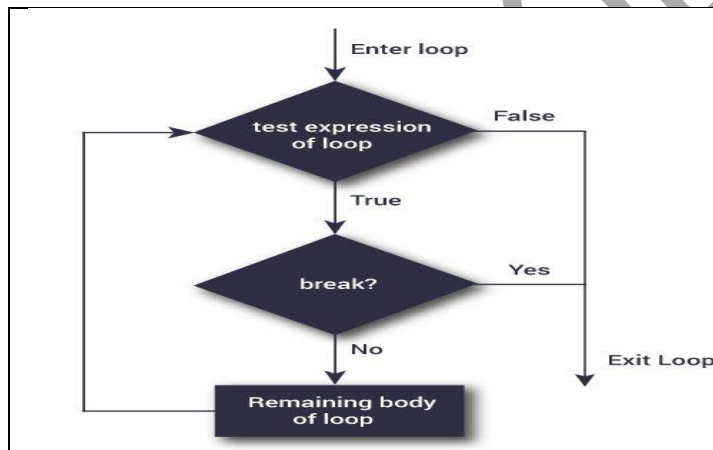
```
rows = 5
for i in range(1, rows + 1):
    for j in range(1, i + 1):
        print(j,end=' ')
    print(' ')
```

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

❖ **break statement**

- ✓ The break statement terminates the loop containing it. Control of the program flows to the statement immediately after the body of the loop.
- ✓ If the break statement is inside a nested loop (loop inside another loop), the break statement will terminate the innermost loop. The break statement can be used in both while and for loops.



Syntax:

break

Example:

```
for i in range(1,11):
    print(i)
    if i == 2:
        break
```

Output:

1

```
i=1
while(i<=10):
    print(i)
    if(i==5):
        break
    i=i+1
```

Output:

1

2

2

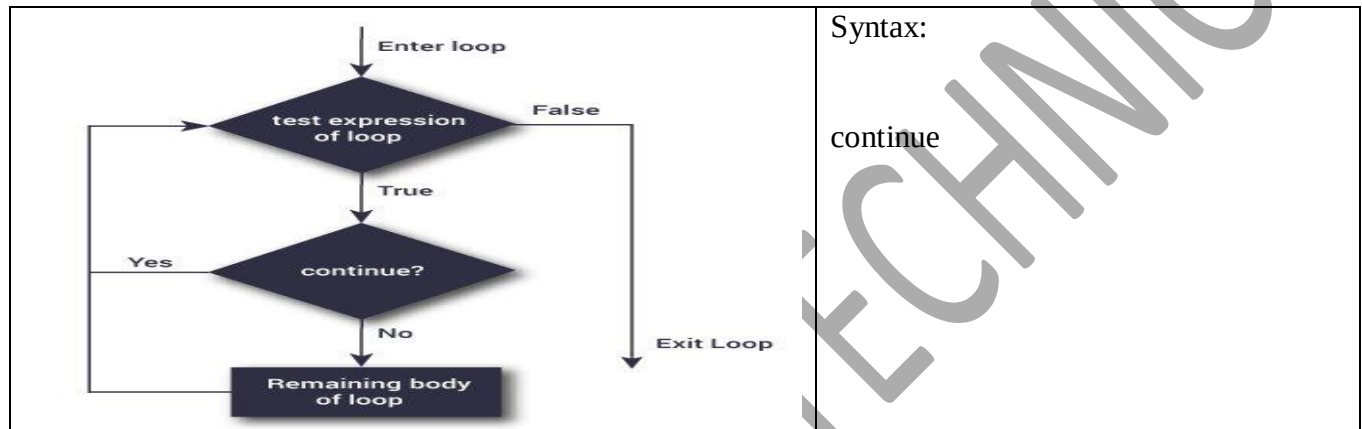
3

4

5

❖ **continue statement**

- ✓ The continue statement in Python returns the control to the beginning of the while loop. The continue statement rejects all the remaining statements in the current iteration of the loop and moves the control back to the top of the loop.
- ✓ The continue statement can be used in both while and for loops.



Syntax:

continue

Example:

```

n=int(input("enter n"))
for i in range(1,n):
    if(i==5):
        continue
    print("%d" %i)
  
```

Output:

```

enter n10
1 2 3 4 6 7 8 9
  
```

```

i=0
while(i<5):
    i=i+1
    if(i==3):
        continue
    print(i)
  
```

Output:

```

1 2 4 5
  
```

❖ **Pass Statement**

- ✓ In Python programming, the pass statement is a null statement.
- ✓ The difference between a comment and a pass statement in Python is that while the interpreter ignores a comment entirely, pass is not ignored.
- ✓ Nothing happens when the pass is executed. It results in no operation (NOP).
- ✓ When the user **does not know** what code to write, so user simply places **pass** at that line. Sometimes, **pass** is used when the user **doesn't want any code to execute**. So user can simply place **pass** where empty code is not allowed, like in loops, function definitions, class definitions, or in if statements. So using pass statement user avoids this error.
- ✓ Syntax:


```
pass
```
- ✓ Example


```

a= {'h', 'e', 'l', 'l', 'o'}
for val in a:
    pass
      
```

Unit 1 Fundamentals of Python

❖ Introduction to Python

- ✓ Python is a widely used general-purpose, high level programming language.
- ✓ Python is interpreted object-oriented language.
- ✓ It is open source software and easily available to use.
- ✓ It enables the straight and easy programming for both small and large applications. It mainly emphasizes on code reusability, readability and using white space.
- ✓ It has features like high-level built-in data structures, dynamic typing, and binding, which makes it attractive for rapid application development.
- ✓ It was mainly developed for emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code.
- ✓ Python is a programming language that lets you work quickly and integrate systems more efficiently.

❖ History of Python

- ✓ It was initially designed by **Guido van Rossum in 1991** and developed by Python Software Foundation.
- ✓ There are two major Python versions- **Python 2 and Python 3**.
- ✓ On 16 October 2000, Python 2.0 was released with many new features.
- ✓ On 3rd December 2008, Python 3.0 was released with more testing and includes new features.

❖ Differences between Compiler and Interpreter

Compiler	Interpreter
The compiler scans the whole program in one go.	Translates the program one statement at a time.
As it scans the code in one go, the errors (if any) are shown at the end together.	Considering it scans code one line at a time, errors are shown line by line.
It converts the source code into object code.	It does not convert source code into object code instead it scans it line by line
It does not require source code for later execution.	It requires source code for later execution.
Execution of the program takes place only after the whole program is compiled.	Execution of the program happens after every line is checked or evaluated.
C, C++, C#, etc are programming languages that are compiler-based.	Python, Ruby, Perl, SNOBOL, MATLAB, etc are programming languages that are interpreter-based.

❖ Differences between Scripting Language and Programming Language:

Scripting Language	Programming Language
A scripting language is a type of programming language designed for a runtime system to automate the execution of tasks.	A programming language is a computer language that is used to communicate with computers using a set of instructions.
It uses an interpreter to convert source code into machine code.	It uses a compiler to convert source code into machine code.

It is interpreted language or interpreter-based language	It is compiled language or compiler-based language
Examples include Perl, PHP, Python, JavaScript, etc.	Examples include C, C++, Java, Python, etc.
Execution of a script takes less time as scripts are generally short.	Execution of a program takes more time since they are compiled.
Do not create a .exe file.	These generate .exe files.
All the scripting languages are programming languages.	All the programming languages are not scripting languages.
There is less maintenance cost.	There is the high maintenance cost.

❖ Python Features

- ✓ **Python is object-oriented:** Structure supports such concepts as polymorphism, operation overloading and multiple inheritance.
- ✓ **Indentation:** Indentation is one of the greatest feature in python
- ✓ **It is free (open source):** Downloading python and installing python is free and easy.
- ✓ **It is Powerful:** It provides Dynamic typing, Built-in types and tools, Library utilities, and Automatic memory management
- ✓ **It is Portable:** Python runs virtually every major platform used today
- ✓ **It is easy to use and learn:** It has no intermediate compile. Python Programs are compiled automatically to an intermediate form called byte code, which the interpreter then reads.
- ✓ **Interpreted Language:** Python is processed at runtime by python Interpreter
- ✓ **Interactive Programming Language:** Users can interact with the python interpreter directly for writing the programs
- ✓ **Straight forward syntax:** The formation of python syntax is simple and straight forward which also makes it popular.

❖ Python Applications

1) Web Applications: We can use Python to develop web applications. It provides libraries to handle internet protocols such as HTML and XML, JSON, Email processing, request, etc. One of Python web-framework named Django is used on Instagram. Python provides many useful frameworks, and these are given below:

- Django and Pyramid framework (Use for heavy applications)
- Flask and Bottle (Micro-framework)

2) Desktop GUI Applications: The GUI stands for the Graphical User Interface, which provides a smooth interaction to any application. Python provides a Tk GUI library to develop a user interface.

3) Console-based Application: Console-based applications run from the command-line or shell. These applications are computer program which are used commands to execute. Python can develop this kind of application very effectively.

4) Software Development: Python is useful for the software development process. It works as a support language and can be used to build control and management, testing, etc.

5) Scientific and Numeric: Python language is the most suitable language for Artificial intelligence or machine learning. It consists of many scientific and mathematical libraries, which makes easy to solve complex calculations.

6) Business Applications: E-commerce and ERP are an example of a business application. This kind of application requires extensively, scalability and readability, and Python provides all these features.

7) Audio or Video-based Applications : Python is flexible to perform multiple tasks and can be used to create multimedia applications. Some multimedia applications which are made by using Python are TimPlayer, cplay, etc.

8) 3D CAD Applications: The CAD (Computer-aided design) is used to design engineering related architecture. It is used to develop the 3D representation of a part of a system. Python can create a 3D CAD application

9) Enterprise Applications: Python can be used to create applications that can be used within an Enterprise or an Organization. Some real-time applications are OpenERP, Tryton, Picalo, etc.

10) Image Processing Application: Python contains many libraries that are used to work with the image. The image can be manipulated according to our requirements. Examples of image processing libraries are: OpenCV, Pillow

❖ Installing Python

There are many interpreters available freely to run Python scripts like IDLE (Integrated Development Environment) which is installed when you install the python software from <http://python.org/downloads/>

Steps to be followed and remembered:

Step 1: Select Version of Python to Install.

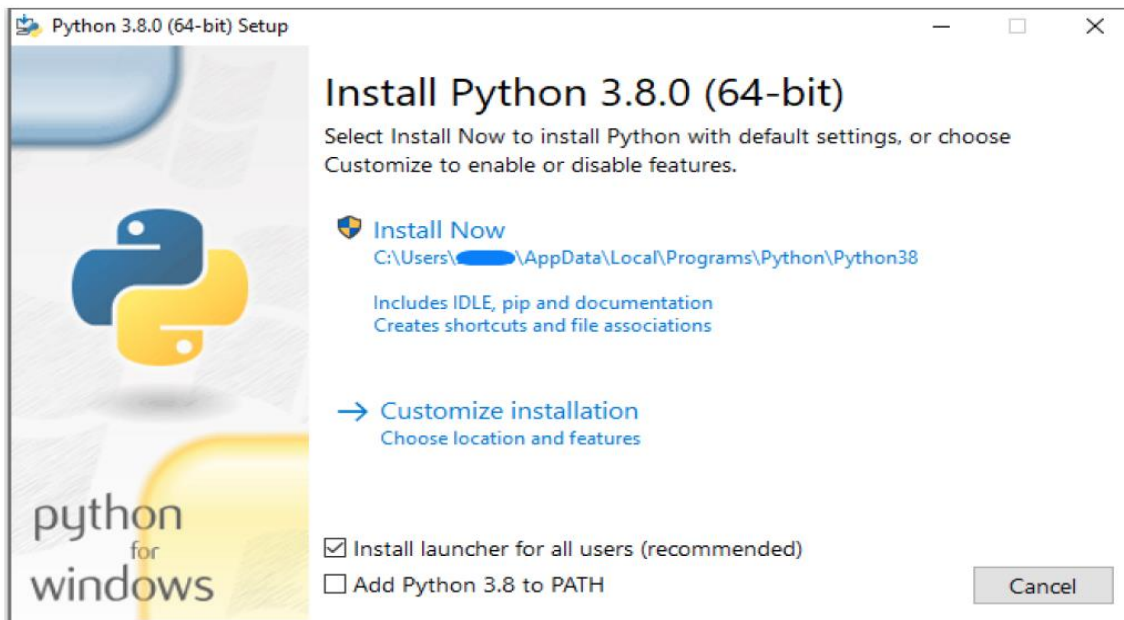
Step 2: Download Python Executable Installer.

Step 3: Run Executable Installer.

Step 4: Verify Python Was Installed On Windows.

Step 5: Verify Pip Was Installed.

Step 6: Add Python Path to Environment Variables (Optional)

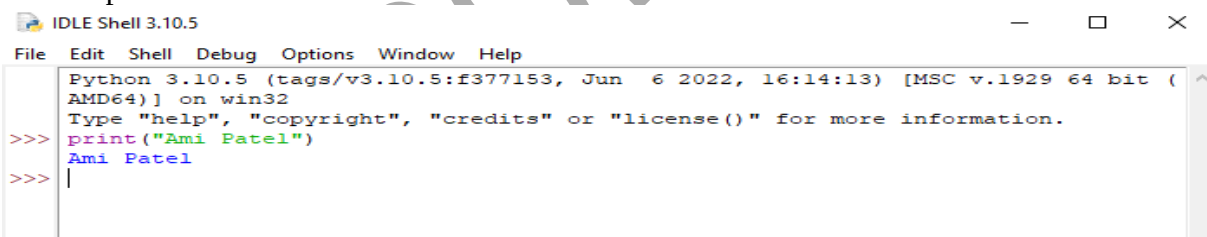


There are two modes for using the Python interpreter:

- ✓ Interactive Mode
- ✓ Script Mode

Running Python in interactive mode:

- ✓ Without passing python script file to the interpreter, directly execute code to Python prompt.
- ✓ Once you're inside the python interpreter, then you can start.
- ✓ `>>> print("Ami Patel")`
Output: Ami Patel



Running Python in script mode:

- ✓ Alternatively, programmers can store Python script source code in a file with the .py extension, and use the interpreter to execute the contents of the file.
- ✓ To execute the script by the interpreter, you have to tell the interpreter the name of the file.

❖ Elements of Python

Python has 4 components

1. **IDLE (Python GUI):** It is a cross platform GUI Integrated Development Environment that is provided with Python for editing and running a Python programs. It is a bundled set of software's tools such as Python Shell for Scripting programs and text editor for creating and modifying Python's source code, an integrated interactive debugger for finding error, help system etc.
2. **Module Docs:** This component allows us to access the python documents such as build in modules, DLLs, libraries, packages etc.

3. **Python (Command line):** Python can also be access from the command line. Command line mode provide very less features in comparison to IDLE but it is fast.

4. **Python Manual :** This component include various documents related to Python such as : installation cum setup guide, tutorials, Python API , FAQs etc.

❖ **Basic Structure of Python program**

The typical structure of a python program include 3 parts:

Import statements // import statements are used to include library files to the python program
Function definitions //This section include the definitions of various functions written in a Python Program
Program statements // This section include the set of statements for solving the given problem.

❖ **Keywords and Identifiers**

Identifiers

- ✓ Identifier is a name given to various programming elements such as a variable, function, class,module or any other object.
- ✓ Following are the rules to create an identifier.
 1. The allowed characters are a-z, A-Z, 0-9 and underscore (_)
 2. It should begin with an alphabet or underscore
 3. It should not be a keyword
 4. It is case sensitive
 5. No blank spaces are allowed.
 6. It can be of any size
- ✓ Valid identifiers examples : si, rate_of_interest, student1, ageStudent
- ✓ Invalid identifier examples : rate of interest, 1student, @age

Keywords

- ✓ Keywords are the identifiers which have a specific meaning in python, there are 33 keywords in python. These may vary from version to version

False	None	True	and
as	assert	break	class
continue	def	del	elif
else	except	finally	for
from	global	if	import
in	is	lambda	nonlocal
not	or	pass	raise
return	try	while	with
yield			

❖ **Variables :** When we create a program, we often need store values so that it can be used in a program. We use variables to store data which can be manipulated by the computer program.

- ✓ Every variable has a name and memory location where it is stored.
- ✓ It Can be of any size
- ✓ Variable name Has allowed characters, which are a-z, A-Z, 0-9 and underscore (_)
- ✓ Variable name should begin with an alphabet or underscore
- ✓ Variable name should not be a keyword
- ✓ Variable name should be meaningful and short
- ✓ Generally, they are written in lower case letters
- ✓ A type or datatype which specify the nature of variable (what type of values can be stored in
- ✓ We can check the type of the variable by using type command
`>>> type(variable_name)`

Assigning Values to Variables:

- ✓ The equal sign (=) is used to assign values to variables.
- ✓ The operand to the left of the = operator is the name of the variable and the operand to the right of the = operator is the value stored in the variable.

For example –

```
a= 100 # An integer assignment
b = 1000.0 # A floating point
c = "Neha" # A string
print (a)
print (b)
print (c)
```

Output:

```
100
1000.0
Neha
```

Multiple Assignment:

Python allows you to assign a single value to several variables simultaneously.

For example :

```
a = b = c = 1
```

Here, an integer object is created with the value 1, and all three variables are assigned to the same memory location. You can also assign multiple objects to multiple variables.

For example –

```
a,b,c = 1,2,"Neha"
```

Here, two integer objects with values 1 and 2 are assigned to variables a and b respectively, and one string object with the value "Neha" is assigned to the variable c.

❖ Data types

1. Number: Number data type stores Numerical Values. These are of three different types:

- a) Integer & Long
- b) Float/floating point
- c) Complex

a) Integers are the whole numbers consisting of + or – sign like 100000, -99, 0, 17.

e.g. age=19

salary=20000

b) Floating Point: Numbers with fractions or decimal point are called floating point numbers. A floating point number will consist of sign (+,-) and a decimal sign(.)

e.g. temperature= -21.9, growth_rate= 0.98333328

The floating point numbers can be represented in scientific notation such as

-2.0X 10⁵ will be represented as -2.0e5

2.0X10⁻⁵ will be 2.0E-5

c) Complex: Complex number is made up of two floating point values, one each for real and imaginary part. For accessing different parts of a variable x we will use x.real and x.imag. Imaginary part of the number is represented by j instead of i, so 1+0j denotes zero imaginary part.

Example

```
>>> x = 1+0j
```

```
>>> print x.real,x.imag
```

```
1.0 0.0
```

2. Boolean: Objects of Boolean type may have one of two values, True or False:

```
>>> type(True)
```

```
<class 'bool'>
```

```
>>> type(False)
```

```
<class 'bool'>
```

3. Sequence: A sequence is an ordered collection of items, indexed by positive integers. Three types of sequence data type available in Python are Strings, Lists & Tuples.

a)String: is an ordered sequence of letters/characters. They are enclosed in single quotes (‘’) or double (‘’’).

Example

```
>>> a = 'CE'
```

```
>>>a="CE"
```

b) Lists: List is also a sequence of values of any type. Values in the list are called elements /items. The items of list are accessed with the help of index (index start from 0). List is enclosed in square brackets.

Example

```
Student = ["Dharmesh", 567, "CS"]
```

c) Tuples: Tuples are a sequence of values of any type, and are indexed by integers. They are immutable i.e. we cannot change the value of items of tuples. Tuples are enclosed in ().

```
Student = ("Dharmesh", 567, "CS")
```

4. Sets

Set is an unordered collection of values, of any type, with no duplicate entry. Sets are immutable.

Example

```
s = set ([1,2,3,4])
```

5. Dictionaries: Dictionaries store a key – value pairs, which are accessed using key. Dictionary is enclosed in curly brackets.

Example

```
d = {1:'a', 2:'b', 3:'c'}
```

6. String: Strings in Python are identified as a contiguous set of characters represented in the quotation marks. Python allows for either pairs of single or double quotes.

- ✓ 'hello' is the same as "hello".
- ✓ Strings can be output to screen using the print function.

For example: print("hello").

```
>>> print("Good Morning")
```

```
mrcet college
```

```
>>> type("Good Morning")
```

```
<class 'str'>
```

Literals: A literal is a constant that appear directly in a program and this value does not change during the program execution i.e. remain fixed.

Example:

```
Num1=5
```

```
Principle_amount= 5000.00
```

Here 5 and 5000.00 are literals. Python support the following literals

- ✓ Numeric literals
- ✓ String Literals
- ✓ Special Literals
- ✓ Collection Literals

❖ Comments:

Single-line comments begins with a hash(#) symbol and is useful in mentioning that the whole line should be considered as a comment until the end of line.

A Multi line comment is useful when we need to comment on many lines. In python, triple double quote(“ “ “) and single quote(‘ ‘ ‘)are used for multi-line commenting.

❖ Type Casting

Type Casting is the method to convert the variable data type into a certain data type in order to the operation required to be performed by users.

There can be two types of Type Casting in Python –

- ✓ Implicit Type Casting
- ✓ Explicit Type Casting

Implicit Type Casting: In this, methods, Python converts data type into another data type automatically. In this process, users don't have to involve in this process.

Example:

```
a = 7      #Python automatically converts a to int
print(type(a))

b = 3.0    #Python automatically converts b to float
print(type(b))

c = a + b   #Python automatically converts c to float as it is a float addition
print(c)
print(type(c))
```

Output:

```
<class 'int'>
<class 'float'>
10.0
<class 'float'>
```

Explicit Type Casting: In this method, Python need user involvement to convert the variable data type into certain data type in order to the operation required.

Mainly in type casting can be done with these data type function:

- **Int()** : Int() function take **float or string** as an argument and return **int** type object.
- **float()** : float() function take **int or string** as an argument and return **float** type object.
- **str()** : str() function take **float or int** as an argument and return **string** type object.

Example:

```
a = 5
n = float(a)      # typecast to float
print(n)
print(type(n))
```

Output:

```
5.0
<class 'float'>
```

❖ Input-Output functions: input, print

How to print/display a statement in Python

A statement is print with the help of print() method.

The syntax to use print() is as follow :

```
>>>print("message to be printed")
```

Or

```
>>>print('message to be printed')
```

Or

```
>>>print(variable_name)
```

Example:

```
x=10
```

```
print(x)
```

```
10
```

Or

```
>>>print("message to be printed", variable_name)
```

Example:

```
>>x=10
```

```
>>print("value of x",x)
```

```
value of x 10
```

Or

```
>>>print('message to be printed', variable_name)
```

Example:

```
>>x=10
```

```
>>print('value of x',x)
```

```
value of x 10
```

How to input a value for the user in Python

A value can be input from the user with the help of input() method the syntax is as follow :

Syntax: variable_name = input("Enter any number")

input method return a string. It can be changed to other datatypes by using type.

Example:

```
age = int(input("Enter your age"))
```

```
percentage= float(input("Enter you percentage"))
```

❖ **Operators:** Operators are special symbols which represents specific operations. They are applied on operand(s), which can be values or variables.

- ✓ **Arithmetic Operators:** Arithmetic operators are used to apply arithmetic operation.

Symbol	Description	Example 1	Example 2
+	Addition	>>>55+45 100	>>> 'Good' + 'Morning' GoodMorning
-	Subtraction	>>>55-45 10	>>>30-80 -50
*	Multiplication	>>>55*45 2475	>>> 'Good'* 3 GoodGoodGood
/	Division	>>>17/5 3 >>>17/5.0 3.4 >>> 17.0/5 3.4	>>>28/3 9
%	Remainder/ Modulo	>>>17%5 2	>>> 23%2 1
**	Exponentiation	>>>2**3 8 >>>16**.5 4.0	>>>2**8 256
//	Integer Division	>>>7.0//2 3.0	>>>3/ / 2 1

- ✓ **Relational Operators :** Relation operators are used to compare two items. The result of relational operator is true or false.

Symbol	Description	Example 1	Example 2
<	Less than	<pre>>>>7<10 True >>> 7<5 False >>> 7<10<15 True >>>7<10 and 10<15 True</pre>	<pre>>>>'Hello'< 'Goodbye' False >>>'Goodbye'< 'Hello' True</pre>
>	Greater than	<pre>>>>7>5 True >>>10<10 False</pre>	<pre>>>>'Hello'> 'Goodbye' True >>>'Goodbye'> 'Hello' False</pre>
<=	less than equal to	<pre>>>> 2<=5 True >>> 7<=4 False</pre>	<pre>>>>'Hello'<= 'Goodbye' False >>>'Goodbye' <= 'Hello' True</pre>
>=	greater than equal to	<pre>>>>10>=10 True >>>10>=12 False</pre>	<pre>>>>'Hello'>= 'Goodbye' True >>>'Goodbye' >= 'Hello' False</pre>
!=, <>	not equal to	<pre>>>>10!=11 True >>>10!=10 False</pre>	<pre>>>>'Hello'!= 'HELLO' True >>>'Hello' != 'Hello' False</pre>
==	equal to	<pre>>>>10==10 True >>>10==11 False</pre>	<pre>>>>'Hello' == 'Hello' True >>>'Hello' == 'Good Bye' False</pre>

Note: Two values that are of different data type will never be equal to each other.

- ✓ **Logical Operators:** Logical Operators give the logical relationship based upon the truth table.

AND			OR			NOT	
Value1	Value2	Result	Value1	Value2	Result	Value1	Result
False	False	False	False	False	False	True	False
False	True	False	False	True	True	False	True
True	False	False	True	False	True		
True	True	True	True	True	True		

Logical Operators

Symbol	Description
or	If any one of the operand is true, then the condition becomes true.
and	If both the operands are true, then the condition becomes true.
not	Reverses the state of operand/condition.

- ✓ **Bitwise Operators** : Bitwise operators are applied upon the bits.

OPERATOR	DESCRIPTION	SYNTAX
&	Bitwise AND	x & y
	Bitwise OR	x y
~	Bitwise NOT	~x
^	Bitwise XOR	x ^ y
>>	Bitwise right shift	x>>
<<	Bitwise left shift	x<<

Bitwise AND operator: Returns 1 if both the bits are 1 else 0.

Example:

a = 10 = 1010 (Binary)

b = 4 = 0100 (Binary)

a & b = 1010

&

0100

= 0000

= 0 (Decimal)

Bitwise or operator: Returns 1 if either of the bit is 1 else 0.

Example:

a = 10 = 1010 (Binary)

b = 4 = 0100 (Binary)

$a \mid b = 1010$

|

0100

= 1110

= 14 (Decimal)

Bitwise not operator: Returns one's complement of the number.

Example:

$a = 10 = 1010$ (Binary)

$\sim a = \sim 1010$

= $-(1010 + 1)$

= $-(1011)$

= -11 (Decimal)

Bitwise xor operator: Returns 1 if one of the bits is 1 and the other is 0 else returns false.

Example:

$a = 10 = 1010$ (Binary)

$b = 4 = 0100$ (Binary)

$a \wedge b = 1010$

^

0100

= 1110

= 14 (Decimal)

Bitwise left shift: Shifts the bits of the number to the left and fills 0 on right as a result. **Example:**

Example

$a = 5 = 0000\ 0101$ (Binary)

$a \ll 1 = 0000\ 1010 = 10$

$a \ll 2 = 0001\ 0100 = 20$

Bitwise right shift: Shifts the bits of the number to the right and fills 0 on left(fills 1 in the case of a negative number) as a result.

Example:

$a = 10 = 0000\ 1010$ (Binary)

$a \gg 1 = 0000\ 0101 = 5$

✓ **Membership Operator:** Membership operators are used to test if a sequence is presented in an object:

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.

Example:1

```
x = ["apple", "orange"]
print("apple" in x)
```

output:

true

Example:2

```
x = ["apple", "orange"]
print("pineapple" not in x)
```

output:

true

✓ Assignment Operators

Assignment operators are used to assign a value to the variable.

=	Assigned values from right side operands to left variable	>>>x=12* >>>y='greetings'	
---	---	------------------------------	--

(*we will use it as initial value of x for following examples)

+=	added and assign back the result to left operand	>>>x+=2	The operand/ expression/ constant written on RHS of operator is will change the value of x to 14
-=	subtracted and assign back the result to left operand	x-=2	x will become 10
=	multiplied and assign back the result to left operand	x=2	x will become 24
/=	divided and assign back the result to left operand	x/=2	x will become 6
%=	taken modulus using two operands and assign the result to left operand	x%=2	x will become 0
=	performed exponential (power) calculation on operators and assign value to the left operand	x=2	x will become 144
//=	performed floor division on operators and assign value to the left operand	x // = 2	x will become 6